

Não sei, só sei que foi assim - Caderno de Códigos

Versão 2025

Contents

1	Main	3
1.1	Template	3
2	Algorithms	3
2.1	All Set Partitions	3
2.2	Binary Search	4
2.3	Compression	4
2.4	Ternary Search	5
3	Data Structures	5
3.1	Bit	5
3.2	Bit 2d	6
3.3	Bit Sort Tree	7
3.4	Disjoint Sparse Table	7
3.5	Merge Sort Tree	7
3.6	Min Stack Queue	8
3.7	Mo	9
3.8	Ordered Set	10
3.9	Seg Tree 2d Iterative	10
3.10	Seg Tree 2d Rec	11
3.11	Seg Tree Iterative	13
3.12	Seg Tree Lazy	14
3.13	Seg Tree Rec	14
3.14	Sparse Table	15
3.15	Sqrt Decomposition	15
3.16	Wavelet Tree	16
4	Geometry	17
4.1	Convex Hull	17
4.2	Point Vector Line	18
4.3	Polygon	20
4.4	Triangle	21
5	Graph	21
5.1	2 Sat	21
5.2	Articulations	22
5.3	Bellman Ford	23
5.4	Bfs	23
5.5	Bfs Grid	24
5.6	Bridges	24
5.7	De Bruijn	25
5.8	Digraph Cycle	25
5.9	Dijkstra	26
5.10	Dinic Max Flow	26
5.11	Dsu	28
5.12	Erdos Gallai	28
5.13	Euler Path Directed	29
5.14	Euler Tour	30
5.15	Ffek Max Flow	31
5.16	Floyd Warshall Path	31
5.17	Held Karp	32
5.18	Hungarian Min Cost Matching	33
5.19	Kruskal	34
5.20	Max Matching Aug	34
5.21	Max Matching Hopcroft Karp	35

5.22	Min Cost Max Flow Dijkstra	36
5.23	Min Cost Max Flow Spfa	38
5.24	Prim	40
5.25	Scc Dag	40
5.26	Spfa	41
5.27	Sucessor Graph	41
5.28	Tarjan	44
6	Math	44
6.1	Binary Matrix Power	44
6.2	Binomial Coefficient Nomod	45
6.3	Digits	46
6.4	Expbin	46
6.5	Extended Gcd	46
6.6	Fibonacci	47
6.7	Fibonacci Factorization	49
6.8	Linear Sieve	49
6.9	Matrix Power	50
6.10	Matrix Power Better	50
6.11	Mod Inverse	51
6.12	Phi	51
6.13	Pollards Rho	52
6.14	Prime Test Sqrt	52
6.15	Segmented Sieve	53
6.16	Sieve	53
6.17	Stirling	55
6.18	Subgrid Count	55
6.19	Sum Of Divisors	56
7	Strings	56
7.1	Aho Corasick	56
7.2	Kmp	57
7.3	Longest Palindromic Prefix	58
7.4	Manacher	58
7.5	Radix Trie	59
7.6	Radix Trie Gpt	60
7.7	Trie	62
8	Trees	62
8.1	All Dist Sum	62
8.2	All Max Dist	63
8.3	Binary Lifting	63
8.4	Centroid	64
8.5	Diameter	64
8.6	Heavy Light Decomp	65
8.7	Lca	66
8.8	Path Sum Update Static	67
8.9	Subtree Queries	68
8.10	Tarjan Offline Lca	68
8.11	Tree Isomorphism	69
9	Examples	71
9.1	Longest Paren Substr	71
9.2	Mo	72
10	Tools	75
10.1	Case Generator	75
10.2	Tree Generator	76

1 Main

1.1 Template

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 #define MAX 212345
5 #define MAXB 2123456
6 #define LOG 20
7 #define PI acos(-1.)
8 #define MAXG 1123
9 #define INF 1012345678 // 10^9
10 #define INFLL 1123456789112345678LL //10^18
11 #define EPS 1e-9
12 #define MOD 1000000007
13 #define MODWPP 998244353
14 typedef long long ll;
15 typedef __int128_t i128;
16 typedef pair<int, int> pii;
17 typedef tuple<int, int, int> tiii;
18
19 int n, m, k, arr[MAXB];
20 ll vet[MAXB];
21 bitset<MAX> b1;
22 vector<vector<int>> graph(MAX);
23
24 int main() {
25     ios::sync_with_stdio(0);
26     cin.tie(0);
27     int a, b, c, t, u, v, w, f, q, ans;
28     ll la, lb, lc, lans;
29     string s1, s2;
30     char ch1, ch2;
31     cin >> t;
32     while(t--) {
33         cin >> n;
34         ans = 0;
35         for(int i = 0; i < n; i++) {
36             cin >> arr[i];
37         }
38         cout << ans << '\n';
39     }
40     return 0;
41 }
```

2 Algorithms

2.1 All Set Partitions

```
1 //gera todas as partições possíveis de n elementos em conjuntos.
2 //complexidade: O(n * B(n)) em que B(N) é o número de Bell de n
3 //que é o número total de jeitos possíveis de fazer as partições.
4 //Ex: para n = 12, deve levar entre 1 e 2 segundos
5
6 //chamar search() com 0 (primeiro elemento a ser considerado)
7
8 vector<vector<int>> part;
9
```

```

10 void search(int i) {
11     if(i == n) {
12         //processa a partição
13         for(auto &a : part) {
14             //processa cada conjunto
15             for(int &b : a) {
16                 //processa cada elemento do conjunto
17             }
18         }
19         return;
20     }
21     for(int j = 0; j < part.size(); j++) {
22         part[j].push_back(i);
23         search(i + 1);
24         part[j].pop_back();
25     }
26     part.push_back({i});
27     search(i + 1);
28     part.pop_back();
29 }

```

2.2 Binary Search

```

1 bool check(ll guess) {
2     //Alguma coisa aqui
3     return 0;
4 }
5
6 ll binSearch(ll inf, ll sup) {
7     ll mid;
8     while(inf < sup) {
9         mid = inf + (sup - inf) / 2;
10        if(check(mid)) sup = mid;
11        else inf = mid + 1;
12    }
13    return inf;
14 }
15
16 // cuidado com overflow
17 // ajustar para 31 caso possa ser utilizado só int mesmo
18 ll maxBinSearch(ll inf, ll sup) {
19     for(ll k = 1LL << 62; k > 0; k >>= 1LL) {
20         if(inf + k <= sup && check(inf + k)) inf += k;
21     }
22     return inf;
23 }
24
25 ll minBinSearch(ll inf, ll sup) {
26     for(ll k = 1LL << 62; k > 0; k >>= 1LL) {
27         if(sup - k >= inf && check(sup - k)) sup -= k;
28     }
29     return sup;
30 }

```

2.3 Compression

```

1 // compressão de números
2 // lê os numeros em arr e escreve devolta em arr seus valores comprimidos

```

```

3
4 void compress() {
5     map<int, int> remap;
6     int idx = 0;
7     for(int i = 0; i < n; i++) {
8         remap[arr[i]] = 0;
9     }
10    for(auto &[key, val] : remap) {
11        val = idx++;
12    }
13    for(int i = 0; i < n; i++) {
14        arr[i] = remap[arr[i]];
15    }
16 }
17
18 // Ou outra alternativa:
19 // preferir essa, é mais cache-friendly
20 void compress() {
21     vector<int> aux(arr, arr + n);
22     sort(aux.begin(), aux.end());
23     aux.erase(unique(aux.begin(), aux.end()), aux.end());
24     for(int i = 0; i < n; i++) {
25         arr[i] = int(lower_bound(aux.begin(), aux.end(), arr[i]) - aux.begin());
26     }
27 }

```

2.4 Ternary Search

```

1 ll check(ll guess) {
2     //Alguma coisa aqui
3     return 0;
4 }
5
6 ll ternarySearch(ll l, ll r) {
7     ll a, b, fa, fb, ans;
8     while(r - l >= 3) {
9         a = l + (r - l) / 3;
10        b = r - (r - l) / 3;
11        fa = check(a);
12        fb = check(b);
13        if(fa < fb) r = b;
14        else l = a;
15    }
16    ans = LLONG_MAX;
17    for(int i = l; i <= r; i++) {
18        ans = min(ans, check(i));
19    }
20    return ans;
21 }

```

3 Data Structures

3.1 Bit

```

1 //Tudo 1-indexed
2
3 ll bit[MAX];
4

```

```

5 void setbit(int i, ll delta) {
6     while(i <= n) {
7         bit[i] += delta;
8         i += i & -i;
9     }
10 }
11
12 ll getbit(int i) {
13     ll ans = 0;
14     while(i) {
15         ans += bit[i];
16         i -= i & -i;
17     }
18     return ans;
19 }
20
21 void build() {
22     int j;
23     for(int i = 1; i <= n; i++) {
24         j = i + (i & -i);
25         if(j <= n) bit[j] += bit[i];
26     }
27 }
28
29 void setrange(int l, int r, ll delta) {
30     setbit(l, delta);
31     setbit(r + 1, -delta);
32 }

```

3.2 Bit 2d

```

1 // tudo 1-indexed
2
3 ll bit[MAXG][MAXG];
4
5 void setbit(int x, int y, int delta) {
6     for(int i = x; i <= n; i += i & -i) {
7         for(int j = y; j <= m; j += j & -j) {
8             bit[i][j] += delta;
9         }
10     }
11 }
12
13
14 ll getbit(int x, int y) {
15     ll ans = 0;
16     for(int i = x; i > 0; i -= i & -i) {
17         for(int j = y; j > 0; j -= j & -j) {
18             ans += bit[i][j];
19         }
20     }
21     return ans;
22 }
23
24 ll rectangle(int i1, int j1, int i2, int j2) {
25     if(i2 > i1) swap(i1, i2);
26     if(j2 > j1) swap(j1, j2);
27     return getbit(i1, j1) - getbit(i1, j2 - 1) - getbit(i2 - 1, j1) + getbit(i2

```

```

        - 1, j2 - 1);
28 }

```

3.3 Bit Sort Tree

```

1 // query retorna para um dado x, quantos números estritamente maiores que x
2 // existem no prefixo
3 // é preciso ordenar cada nó da bit antes de usar!
4
5 vector<vector<int>> bit(MAX);
6
7 void insertBit(int i, int x) {
8     while(i <= n) {
9         bit[i].push_back(x);
10        i += i & -i;
11    }
12 }
13
14 int query(int i, int x) {
15     int ans = 0;
16     while(i) {
17         ans += bit[i].end() - upper_bound(bit[i].begin(), bit[i].end(), x);
18         i -= i & -i;
19     }
20     return ans;
21 }

```

3.4 Disjoint Sparse Table

```

1 //queries 0-indexed
2 //valores iniciais em arr[]
3
4 int n2, table[2 * MAX][LOG];
5
6 void build() {
7     n2 = n;
8     while(n2 & (n2 - 1)) n2++;
9     for(int j = 0; (1 << j) < n2; j++) {
10        int len = 1 << j;
11        for(int k = len; k < n2; k += 2 * len) {
12            table[k][j] = arr[k];
13            table[k - 1][j] = arr[k - 1];
14            for(int i = k + 1; i < k + len; i++) table[i][j] = table[i - 1][j] ^
                arr[i];
15            for(int i = k - 2; i >= k - len; i--) table[i][j] = arr[i] ^ table[i
                + 1][j];
16        }
17    }
18 }
19
20 int query(int l, int r) {
21     if(l == r) return arr[l];
22     int j = __builtin_clz(1) - __builtin_clz(l ^ r);
23     return table[l][j] ^ table[r][j];
24 }

```

3.5 Merge Sort Tree

```

1 // build: O(n log (n))

```

```

2 // não esquecer de usar multiset ao invés de set, se tiver que fazer algo que
  suporte updates
3
4 #define lft(u) ((u) << 1)
5 #define rgt(u) (lft(u) + 1)
6
7 vector<int> st[2 * MAX];
8
9 void build() {
10     for(int i = n; i < (n << 1); i++) {
11         st[i] = {arr[i - n]};
12     }
13     for(int i = n - 1; i > 0; i--) {
14         merge(st[lft(i)].begin(), st[lft(i)].end(),
15             st[rgt(i)].begin(), st[rgt(i)].end(),
16             back_inserter(st[i])
17         );
18     }
19 }
20
21 // conta quantos números entre x e y, inclusive, existem no intervalo [l, r]
22 int countRange(int l, int r, int x, int y) {
23     int ans = 0;
24     for(l += n, r += n; l <= r; ++l >>= 1, --r >>= 1) {
25         if((l & 1) == 1) ans += upper_bound(st[l].begin(), st[l].end(), y) -
            lower_bound(st[l].begin(), st[l].end(), x);
26         if((r & 1) == 0) ans += upper_bound(st[r].begin(), st[r].end(), y) -
            lower_bound(st[r].begin(), st[r].end(), x);
27     }
28     return ans;
29 }
30
31 // retorna qual o menor elemento maior ou igual a x dentro de [l, r]
32 int minGteX(int l, int r, int x) {
33     int ans = INF;
34     for(l += n, r += n; l <= r; ++l >>= 1, --r >>= 1) {
35         if((l & 1) == 1) {
36             auto it1 = lower_bound(st[l].begin(), st[l].end(), x);
37             if(it1 != st[l].end()) ans = min(ans, *it1);
38         }
39         if((r & 1) == 0) {
40             auto it2 = lower_bound(st[r].begin(), st[r].end(), x);
41             if(it2 != st[r].end()) ans = min(ans, *it2);
42         }
43     }
44     return ans;
45 }

```

3.6 Min Stack Queue

```

1 // simulação de queue com duas stacks
2 // é a melhor escolha para resolver o problema:
3 // "encontrar o mínimo de todos os subarrays de tamanho k"
4 // inserir os primeiros k números, pegar o mínimo, e para cada novo elemento i
5 // só chamar insertStk(arr[i]) e chamar removeStk()
6
7 stack<pii> stk1, stk2;
8

```



```

9 void insertStk(int x) {
10     int y = stk1.empty() ? x : min(x, stk1.top().second);
11     stk1.emplace(x, y);
12 }
13
14 void removeStk() {
15     if(stk2.empty()) {
16         while(!stk1.empty()) {
17             int x = stk1.top().first;
18             stk1.pop();
19             int y = stk2.empty() ? x : min(x, stk2.top().second);
20             stk2.emplace(x, y);
21         }
22     }
23     stk2.pop();
24 }
25
26 int findMinStk() {
27     if(stk1.empty() || stk2.empty()) {
28         return stk1.empty() ? stk2.top().second : stk1.top().second;
29     } else return min(stk1.top().second, stk2.top().second);
30 }

```

3.7 Mo

```

1 // Algoritmo de Mo para responder queries difíceis sobre segmentos delimitados
  // por l e r arbitrários
2 // tudo 0-indexed
3 // k deve conter a quantidade de queries
4 // inserir no vetor queries do como (l, r, i), l e r os limites e i o índice da
  // query
5 // resposta para cada query na ordem da entrada estará em ans
6 // modificar a estrutura e lógica de add/remove para o que o problema pedir
7 // BSIZE = sqrt(MAX)
8
9 // no exemplo abaixo, o algoritmo é utilizado para responder as seguintes
  // queries:
10 // dado um subarray delimitado por l e r, responder quantos números distintos
  // existem nesse intervalo
11
12 int ans[MAX], occ[MAX];
13 const int BSIZE = 450;
14 vector<tiii> queries(MAX);
15
16 bool comp(const tiii a, const tiii b) {
17     auto [al, ar, ai] = a;
18     auto [bl, br, bi] = b;
19     if(al / BSIZE != bl / BSIZE) return a < b;
20     return (al / BSIZE & 1) ? (ar < br) : (ar > br);
21 }
22
23 void remove(int i) {
24     occ[arr[i]]--;
25     if(occ[arr[i]] == 0) m--;
26 }
27
28 void add(int i) {
29     occ[arr[i]]++;

```

```

30     if(occ[arr[i]] == 1) m++;
31 }
32
33 int getAns() {
34     return m;
35 }
36
37 void mo() {
38     sort(queries.begin(), queries.begin() + k, comp);
39     int l = 0, r = -1;
40     for(int i = 0; i < k; i++) {
41         auto &[ql, qr, j] = queries[i];
42         while(l > ql) {
43             l--;
44             add(l);
45         }
46         while(r < qr) {
47             r++;
48             add(r);
49         }
50         while(l < ql) {
51             remove(l);
52             l++;
53         }
54         while(r > qr) {
55             remove(r);
56             r--;
57         }
58         ans[j] = getAns();
59     }
60 }

```

3.8 Ordered Set

```

1  #include <ext/pb_ds/assoc_container.hpp>
2  #include <ext/pb_ds/tree_policy.hpp>
3  using namespace __gnu_pbds;
4  #define ordered_set tree<int, null_type, less<int>, rb_tree_tag,
   tree_order_statistics_node_update>
5
6
7  //LEMBRANDO QUE ISSO É UM SET
8  //SE PRECISAR SER UM MULTISSET, TEM QUE MUDAR O COMPARADOR PARA LESS_EQUAL
9  //OU CADA ELEMENTO SER UM PAR

```

3.9 Seg Tree 2d Iterative

```

1  // Exemplo de uso com manhattan distance
2  // Coordenadas dos pontos nesse exemplo vão de -500 a 500
3  // Como eles podem ser negativo, chamar as funções somando 500 em cada
4  // coordenada, para indexar tudo em 0
5  // Tamanho do grid deve ser (qtd_diagonais)^2
6
7  #define MAX 2123
8  #define left(p) (p << 1)
9  #define right(p) (left(p) + 1)
10
11 int st[2 * MAX][2 * MAX];
12

```

```

13 int gcd(int a, int b) {
14     if(b == 0) return a;
15     return gcd(b, a % b);
16 }
17
18 void build() {
19     for(int x = 2 * n; x; x--) {
20         for(int y = 2 * n; y; y--) {
21             if(x < n) st[x][y] = gcd(st[left(x)][y], st[right(x)][y]);
22             if(y < n) st[x][y] = gcd(st[x][left(y)], st[x][right(y)]);
23         }
24     }
25 }
26
27 int query(int x1, int y1, int x2, int y2) {
28     int ret = 0, y3 = y1 + n, y4 = y2 + n;
29     for(x1 += n, x2 += n; x1 <= x2; (++x1) >>= 1, (--x2) >>= 1) {
30         for(y1 = y3, y2 = y4; y1 <= y2; (++y1) >>= 1, (--y2) >>= 1) {
31             if(ret == 1) return ret;
32             if((x1 & 1) && (y1 & 1)) ret = gcd(ret, st[x1][y1]);
33             if((x1 & 1) && !(y2 & 1)) ret = gcd(ret, st[x1][y2]);
34             if(!(x2 & 1) && (y1 & 1)) ret = gcd(ret, st[x2][y1]);
35             if(!(x2 & 1) && !(y2 & 1)) ret = gcd(ret, st[x2][y2]);
36         }
37     }
38     return ret;
39 }
40
41 void update(int x, int y, int val) {
42     int y2 = y + n;
43     for(x += n; x; x >>= 1, y = y2) {
44         if(x >= n) st[x][y] = val;
45         else st[x][y] = gcd(st[left(x)][y], st[right(x)][y]);
46         while(y >>= 1) st[x][y] = gcd(st[x][left(y)], st[x][right(y)]);
47     }
48 }
49
50 void insert_val(int x, int y, int val) {
51     st[x + n][y + n] = val;
52 }
53
54 int manhattan_query(int x, int y, int d) {
55     int nx = x + y, ny = x - y + 1000;
56     return query(max(nx - d, 0), max(ny - d, 0), min(nx + d, n), min(ny + d, n))
57     ;
58 }
59
60 void manhattan_update(int x, int y, int val) {
61     int nx = x + y, ny = x - y + 1000;
62     update(nx, ny, val);
63 }

```

3.10 Seg Tree 2d Rec

```

1 // Exemplo de uso com manhattan distance
2 // Coordenadas dos pontos nesse exemplo vão de -500 a 500
3 // Como eles podem ser negativo, chamar as funções somando 500 em cada
4 // coordenada, para indexar tudo em 0

```

```

5 // Tamanho do grid deve ser (qtd_diagonais)^2
6
7 // Se não for query de manhattan distance, n = linhas e m = colunas
8
9 #define MAX 2123
10 #define left(u) ((u) + 1)
11 #define right(u, l, m) ((u) + 2 * ((m) - (l) + 1))
12
13 int a[MAX][MAX], t[2 * MAX][2 * MAX];
14
15 int gcd(int a, int b) {
16     return b == 0 ? a : gcd(b, a % b);
17 }
18
19 void build_y(int vx, int lx, int rx, int vy, int ly, int ry) {
20     if(ly == ry) {
21         if(lx == rx) t[vx][vy] = a[lx][ly];
22         else t[vx][vy] = gcd(t[left(vx)][vy], t[right(vx, lx, (lx + rx) / 2)][vy]);
23     } else {
24         int my = (ly + ry) / 2;
25         build_y(vx, lx, rx, left(vy), ly, my);
26         build_y(vx, lx, rx, right(vy, ly, my), my + 1, ry);
27         t[vx][vy] = gcd(t[vx][left(vy)], t[vx][right(vy, ly, my)]);
28     }
29 }
30
31 void build_x(int vx, int lx, int rx) {
32     if(lx != rx) {
33         int mx = (lx + rx) / 2;
34         build_x(left(vx), lx, mx);
35         build_x(right(vx, lx, mx), mx + 1, rx);
36     }
37     build_y(vx, lx, rx, 0, 0, m - 1);
38 }
39
40 int query_y(int vx, int vy, int tly, int try_, int ly, int ry) {
41     if(ly > ry) return 0;
42     if(ly == tly && try_ == ry) return t[vx][vy];
43     int tmy = (tly + try_) / 2;
44     return gcd(
45         query_y(vx, left(vy), tly, tmy, ly, min(ry, tmy)),
46         query_y(vx, right(vy, tly, tmy), tmy + 1, try_, max(ly, tmy + 1), ry)
47     );
48 }
49
50 int query_x(int vx, int tlx, int trx, int lx, int rx, int ly, int ry) {
51     if(lx > rx) return 0;
52     if(lx == tlx && trx == rx) return query_y(vx, 0, 0, m - 1, ly, ry);
53     int tmx = (tlx + trx) / 2;
54     return gcd(
55         query_x(left(vx), tlx, tmx, lx, min(rx, tmx), ly, ry),
56         query_x(right(vx, tlx, tmx), tmx + 1, trx, max(lx, tmx + 1), rx, ly, ry)
57     );
58 }
59

```

```

60 void update_y(int vx, int lx, int rx, int vy, int ly, int ry, int x, int y, int
    val) {
61     if(ly == ry) {
62         if(lx == rx) t[vx][vy] = val;
63         else t[vx][vy] = gcd(t[left(vx)][vy], t[right(vx, lx, (lx + rx) / 2)][vy
            ]);
64     } else {
65         int my = (ly + ry) / 2;
66         if(y <= my) update_y(vx, lx, rx, left(vy), ly, my, x, y, val);
67         else update_y(vx, lx, rx, right(vy, ly, my), my + 1, ry, x, y, val);
68         t[vx][vy] = gcd(t[vx][left(vy)], t[vx][right(vy, ly, my)]);
69     }
70 }
71
72 void update_x(int vx, int lx, int rx, int x, int y, int val) {
73     if(lx != rx) {
74         int mx = (lx + rx) / 2;
75         if(x <= mx) update_x(left(vx), lx, mx, x, y, val);
76         else update_x(right(vx, lx, mx), mx + 1, rx, x, y, val);
77     }
78     update_y(vx, lx, rx, 0, 0, m - 1, x, y, val);
79 }
80
81 int manhattan_query(int x, int y, int d) {
82     int nx = x + y;
83     int ny = x - y + 1050;
84     return query_x(0, 0, n - 1, max(nx - d, 0), min(nx + d, n - 1), max(ny - d,
        0), min(ny + d, m - 1));
85 }
86
87 void manhattan_update(int x, int y, int val) {
88     int nx = x + y;
89     int ny = x - y + 1050;
90     update_x(0, 0, n - 1, nx, ny, val);
91 }

```

3.11 Seg Tree Iterative

```

1 // Operações 0-indexed
2
3 #define left(u) (2 * (u))
4 #define right(u) (left(u) + 1)
5
6 int st[2 * MAX];
7
8 void build() {
9     for(int i = n - 1; i > 0; i--) {
10         st[i] = st[left(i)] + st[right(i)];
11     }
12 }
13
14 int query(int l, int r) {
15     int ans = 0;
16     for(l += n, r += n; l <= r; ++l /= 2, --r /= 2) {
17         if(l % 2 == 1) ans += st[l];
18         if(r % 2 == 0) ans += st[r];
19     }

```

```

20     return ans;
21 }
22
23 void update(int u, int x) {
24     st[u += n] = x;
25     while(u /= 2) st[u] = st[left(u)] + st[right(u)];
26 }

```

3.12 Seg Tree Lazy

```

1  #define left(u) ((u) + 1)
2  #define right(u, m, l) ((u) + 2 * ((m) - (l) + 1))
3
4  int st[2 * MAX], lazy[2 * MAX];
5
6  void update_lazy(int u, int l, int r, int val) {
7      int m = (l + r) / 2;
8      int k = r - l + 1;
9      st[u] += k * val;
10     if(l < r) {
11         lazy[left(u)] += val;
12         lazy[right(u, m, l)] += val;
13     }
14     lazy[u] = 0;
15 }
16
17 int build_st(int u, int l, int r) {
18     int m = (l + r) / 2;
19     if(l == r) return st[u] = arr[l];
20     return st[u] = build_st(left(u), l, m) + build_st(right(u, m, l), m + 1, r);
21 }
22
23 int query_st(int u, int l, int r, int i, int j) {
24     if(lazy[u] < 0) update_lazy(u, l, r, lazy[u]);
25     if(r < i || l > j) return 0;
26     if(i <= l && r <= j) return st[u];
27     int m = (l + r) / 2;
28     return query_st(left(u), l, m, i, j) + query_st(right(u, m, l), m + 1, r, i,
29         j);
30 }
31
32 int update_range(int u, int l, int r, int i, int j, int val) {
33     if(lazy[u] < 0) update_lazy(u, l, r, lazy[u]);
34     if(r < i || l > j) return st[u];
35     if(i <= l && r <= j) {
36         update_lazy(u, l, r, val);
37         return st[u];
38     }
39     int m = (l + r) / 2;
40     return st[u] = update_range(left(u), l, m, i, j, val) + update_range(right(u,
    m, l), m + 1, r, i, j, val);

```

3.13 Seg Tree Rec

```

1  #define left(u) ((u) + 1)
2  #define right(u, m, l) ((u) + 2 * ((m) - (l) + 1))
3

```

```

4 ll st[2 * MAX];
5
6 ll build_st(int u, int l, int r) {
7     int m = (l + r) / 2;
8     if(l == r) return st[u] = arr[l];
9     return st[u] = build_st(left(u), l, m) + build_st(right(u, m, l), m + 1, r);
10 }
11
12 ll query_st(int u, int l, int r, int i, int j) {
13     if(r < i || l > j) return 0;
14     if(i <= l && r <= j) return st[u];
15     int m = (l + r) / 2;
16     return query_st(left(u), l, m, i, j) + query_st(right(u, m, l), m + 1, r, i,
17         j);
18 }
19 ll update_st(int u, int l, int r, int i, ll val) {
20     if(i < l || i > r) return st[u];
21     if(i == l && l == r) return st[u] = val;
22     int m = (l + r) / 2;
23     return st[u] = update_st(left(u), l, m, i, val) + update_st(right(u, m, l),
24         m + 1, r, i, val);
25 }
26 //Tudo 0-indexed
27 build_st(0, 0, n - 1);
28 update_st(0, 0, n - 1, i, j);
29 query_st(0, 0, n - 1, i, j)

```

3.14 Sparse Table

```

1 //queries 0-indexed
2 //valores iniciais em arr[]
3
4 int table[MAX][LOG];
5
6 void build() {
7     for(int i = 0; i < n; i++) table[i][0] = arr[i];
8     for(int j = 1; j < LOG; j++) {
9         for(int i = 0; i + (1 << j) - 1 < n; i++) {
10             table[i][j] = min(table[i][j - 1], table[i + (1 << (j - 1))][j - 1])
11             ;
12         }
13     }
14 }
15 int query(int l, int r) {
16     int j = __builtin_clz(1) - __builtin_clz(r - l + 1);
17     return min(table[l][j], table[r - (1 << j) + 1][j]);
18 }

```

3.15 Sqrt Decomposition

```

1 // valores iniciais em arr
2 // tudo 0-indexed
3 // BSIZE é raiz quadrada de MAX
4 // query em O(sqrt(n)), update em O(1)

```

```

5 // caso haja um fator log envolvido na resposta das queries, o BSIZE ótimo é
  sqrt(n / log(n))
6
7 const int BSIZE = 450;
8 ll block[BSIZE];
9
10 void preprocess() {
11     for(int i = 0; i < n; i++) {
12         block[i / BSIZE] += arr[i];
13     }
14 }
15
16 ll query(int l, int r) {
17     int bl = l / BSIZE, br = r / BSIZE;
18     ll ans = 0;
19     if(bl == br) {
20         for(int i = l; i <= r; i++) ans += arr[i];
21     } else {
22         for(int i = l; i < (bl + 1) * BSIZE; i++) ans += arr[i];
23         for(int i = bl + 1; i < br; i++) ans += block[i];
24         for(int i = br * BSIZE; i <= r; i++) ans += arr[i];
25     }
26     return ans;
27 }
28
29 void update(int i, ll x) {
30     int b = i / BSIZE;
31     block[b] += x - arr[i];
32     arr[i] = x;
33 }

```

3.16 Wavelet Tree

```

1 // Wavelet Tree
2 //
3 // Usa  $O(\sigma + n \log(\sigma))$  de memoria,
4 // onde  $\sigma = \text{MAXN} - \text{MINN}$ 
5 // Depois do build, o v fica ordenado
6 // count(i, j, x, y) retorna o numero de elementos de
7 // v[i, j) que pertencem a [x, y]
8 // kth(i, j, k) retorna o elemento que estaria
9 // na posicao k-1 de v[i, j), se ele fosse ordenado
10 // sum(i, j, x, y) retorna a soma dos elementos de
11 // v[i, j) que pertencem a [x, y]
12 // sumk(i, j, k) retorna a soma dos k-esimos menores
13 // elementos de v[i, j) (sum(i, j, 1) retorna o menor)
14
15 vector<int> lft[4 * MAX], pref[4 * MAX];
16 int sigma;
17
18 void build(int b = 0, int e = n, int p = 1, int l = 0, int r = sigma - 1) {
19     int m = (l + r) / 2; lft[p].push_back(0); pref[p].push_back(0);
20     for(int i = b; i < e; i++) {
21         lft[p].push_back(lft[p].back() + (arr[i] <= m));
22         pref[p].push_back(pref[p].back() + arr[i]);
23     }
24     if(l == r) return;
25     int m2 = stable_partition(arr + b, arr + e, [=](int i){return i <= m;}) -

```



```

    arr;
26    build(b, m2, 2 * p, l, m), build(m2, e, 2 * p + 1, m + 1, r);
27 }
28
29 int count(int i, int j, int x, int y, int p = 1, int l = 0, int r = sigma - 1) {
30     if(y < l or r < x) return 0;
31     if(x <= l and r <= y) return j - i;
32     int m = (l + r) / 2, ei = lft[p][i], ej = lft[p][j];
33     return count(ei, ej, x, y, 2 * p, l, m) + count(i - ei, j - ej, x, y, 2 * p
        + 1, m + 1, r);
34 }
35
36 int kth(int i, int j, int k, int p = 1, int l = 0, int r = sigma - 1) {
37     if(l == r) return l;
38     int m = (l + r) / 2, ei = lft[p][i], ej = lft[p][j];
39     if(k <= ej - ei) return kth(ei, ej, k, 2 * p, l, m);
40     return kth(i - ei, j - ej, k - (ej - ei), 2 * p + 1, m + 1, r);
41 }
42
43 int sum(int i, int j, int x, int y, int p = 1, int l = 0, int r = sigma - 1) {
44     if(y < l or r < x) return 0;
45     if(x <= l and r <= y) return pref[p][j] - pref[p][i];
46     int m = (l + r) / 2, ei = lft[p][i], ej = lft[p][j];
47     return sum(ei, ej, x, y, 2 * p, l, m) + sum(i - ei, j - ej, x, y, 2 * p + 1,
        m + 1, r);
48 }
49
50 int sumk(int i, int j, int k, int p = 1, int l = 0, int r = sigma - 1) {
51     if(l == r) return l * k;
52     int m = (l + r) / 2, ei = lft[p][i], ej = lft[p][j];
53     if(k <= ej - ei) return sumk(ei, ej, k, 2 * p, l, m);
54     return pref[2 * p][ej] - pref[2 * p][ei] + sumk(i - ei, j - ej, k - (ej - ei
        ), 2 * p + 1, m + 1, r);
55 }

```

4 Geometry

4.1 Convex Hull

```

1  struct point {
2      int x, y;
3      point() {}
4      point(int _x, int _y) : x(_x), y(_y) {}
5  };
6
7  struct vec {
8      int x, y;
9      vec() {}
10     vec(int _x, int _y) : x(_x), y(_y) {}
11     vec(point a, point b) : x(b.x - a.x), y(b.y - a.y) {}
12 };
13
14 bool operator<(point a, point b) {
15     return a.x < b.x || (a.x == b.x && a.y < b.y);
16 }
17
18 point p[MAX], hull[2 * MAX];

```

```

19
20 int cross(vec u, vec v) {
21     return (u.x * v.y - u.y * v.x) < 0;
22 }
23
24 void convexHull() {
25     k = 0;
26     sort(p, p + n);
27     for(int i = 0; i < n; i++) {
28         while(k >= 2 && !cross(vec(hull[k - 2], p[i]), vec(hull[k - 2], hull[k -
29             1]))) {
30             k--;
31         }
32         hull[k++] = p[i];
33     }
34     for(int i = n - 2, t = k + 1; i >= 0; i--) {
35         while(k >= t && !cross(vec(hull[k - 2], p[i]), vec(hull[k - 2], hull[k -
36             1]))) {
37             k--;
38         }
39         hull[k++] = p[i];
40     }
41 }

```

4.2 Point Vector Line

```

1 // Cuidado com overflow nas operações intermediárias
2 // Cuidado com vetor diretor da reta ser 0
3
4 #define PI acos(-1.)
5 #define rad(t) ((t) * (PI / 180.))
6
7 struct point {
8     int x, y;
9     point() {}
10    point(int _x, int _y) : x(_x), y(_y) {}
11 };
12
13 struct vec {
14     int x, y;
15     vec() {}
16     vec(int _x, int _y) : x(_x), y(_y) {}
17     vec(point a, point b) : x(b.x - a.x), y(b.y - a.y) {}
18 };
19
20 // Forma paramétrica
21 struct line {
22     point p;
23     vec dir;
24     line() {}
25     line(point _p, vec _dir) : p(_p), dir(_dir) {}
26     line(point a, point b) : p(a), dir(a, b) {}
27 };
28
29 struct seg {
30     point a, b;
31     vec u;
32     seg() {}

```

```

33     seg(point _a, point _b) : a(_a), b(_b), u(_a, _b) {}
34 };
35
36 //cuidado que o retorno aqui pode ser negativo. isso calcula a area do
    paralelogramo quando
37 //passado o modulo no resultado, ou seja abs(cross(u, v)). Para ter a area do
    triangulo,
38 //multiplicar esse abs do retorno por 0.5 e cuidado com inteiros e doubles.
39 int cross(vec u, vec v) {
40     return u.x * v.y - u.y * v.x;
41 }
42
43 //o dot product de um vetor com ele mesmo resulta no quadrado da norma
44 int dot(vec u, vec v) {
45     return u.x * v.x + u.y * v.y;
46 }
47
48 bool onLine(line l, point p) {
49     return cross(l.dir, vec(l.p, p)) == 0;
50 }
51
52 bool parallel(line l, line r) {
53     return cross(l.dir, r.dir) == 0;
54 }
55
56 double linesDist(line l, line r){
57     if(!parallel(l, r)) return 0.0;
58     vec v(l.p, r.p);
59     double area2 = fabs(cross(l.dir, v)); // 2 * area
60     double len = sqrt(l.dir.x * l.dir.x + l.dir.y * l.dir.y);
61     return area2 / len;
62 }
63
64 double linePointDist(line l, point p){
65     vec v(l.p, p);
66     double area2 = fabs(cross(l.dir, v)); // 2 * area
67     double len = sqrt(l.dir.x * l.dir.x + l.dir.y * l.dir.y);
68     return area2 / len;
69 }
70
71 int side(line l, point p) {
72     return cross(l.dir, vec(l.p, p));
73     // > 0 esq, < 0 dir, 0 = na reta
74 }
75
76 bool intersection(line l, line r, point &res) {
77     double d = cross(l.dir, r.dir);
78     if(fabs(d) < EPS) return false;
79     vec delta(l.p, r.p);
80     double t = cross(delta, r.dir) / d;
81     res.x = l.p.x + t * l.dir.x;
82     res.y = l.p.y + t * l.dir.y;
83     return true;
84 }
85
86 // deg em graus, cuidado se ja estiver em radianos
87 vec rotateVec(vec u, double deg) {

```

```

88     double t = rad(deg);
89     double x = cos(t) * u.x + (-sin(t) * u.y);
90     double y = sin(t) * u.x + cos(t) * u.y;
91     return vec(x, y);
92 }
93
94 bool operator<(point a, point b) {
95     return a.x < b.x || (a.x == b.x && a.y < b.y);
96 }
97
98 //Cuidado com reta de tamanho 0, ali na divisão por len
99 //nesse caso, é uma distancia entre dois pontos normal
100 //Todas as structs e funções envolvidas aqui tem que ser double
101 double pointSegDist(seg s, point p) {
102     point cp;
103     vec u = vec(s.a, p);
104     double normSq = dot(s.u, s.u);
105     if(normSq < EPS) return sqrt(dot(vec(p, s.a), vec(p, s.a)));
106     double l = dot(s.u, u) / normSq;
107     if(l < 0.) cp = s.a;
108     else if(l > 1.) cp = s.b;
109     else {
110         cp.x = s.a.x + l * s.u.x;
111         cp.y = s.a.y + l * s.u.y;
112     }
113     return sqrt(dot(vec(cp, p), vec(cp, p)));
114 }
115
116 //Funciona somente para segmentos não colineares
117 bool segIntersect(seg s, seg r) {
118     return cross(s.u, vec(s.a, r.a)) * cross(s.u, vec(s.a, r.b)) < EPS &&
119         cross(r.u, vec(r.a, s.a)) * cross(r.u, vec(r.a, s.b)) < EPS;
120 }
121
122 bool collinearSeg(seg s, seg r) {
123     return fabs(cross(s.u, vec(s.a, r.a))) < EPS &&
124         fabs(cross(s.u, vec(s.a, r.b))) < EPS;
125 }
126
127 double segDist(seg s, seg r) {
128     if(!collinearSeg(s, r) && segIntersect(s, r)) return 0.;
129     return min({pointSegDist(r, s.a),
130         pointSegDist(r, s.b),
131         pointSegDist(s, r.a),
132         pointSegDist(s, r.b)});
133 }

```

4.3 Polygon

```

1 //Calcula a area do polígono simples, convexo ou nao
2 //vetor de pontos do parâmetro deve conter a lista de pontos do polígono
3 //nenhuma posição sobrando, e sem repetições de pontos
4 //point aqui deve ter as coordenadas long long
5 //os vértices devem estar ordenados no sentido horário ou anti-horário,
6 //se não, não funciona
7
8 double polArea(vector<point> &p) {
9     p.emplace_back(p[0]); //repete o primeiro ponto no final

```

```

10     ll ans = 0;
11     for(int i = 0; i < (int) p.size() - 1; i++) {
12         ans += p[i].x * p[i + 1].y - p[i].y * p[i + 1].x;
13     }
14     return 0.5 * llabs(ans); //llabs(ans) conterá 2 * area
15 }

```

4.4 Triangle

```

1 // fórmula área do triângulo, incírculo e circuncírculo
2 typedef tuple<double, double, double> tddd;
3
4 tddd calc(double a, double b, double c) {
5     double s = (a + b + c) / 2;
6     double tri = sqrt(s * (s - a) * (s - b) * (s - c));
7     double r = tri / s;
8     double inCirc = r * r * PI;
9     r = a * b * c / (4 * tri);
10    double circC = r * r * PI;
11    return make_tuple(tri, inCirc, circC);
12 }

```

5 Graph

5.1 2 Sat

```

1 // Resolvedor de 2-SAT, retorno 1 = há solução, 0 = não há
2
3 // A XOR B = (A ^ B) = (A | B) & (-A | -B) // colocar as 4 arestas para as duas
   disjunções
4 // Uma cláusula A XOR B significa uma obrigatoriedade de A e B serem diferentes
5
6 // A <-> B = (-A ^ B)
7 // A <-> B = (A -> B) & (-A -> -B)
8 // Uma cláusula A <-> B (A se e somente se B) significa uma obrigatoriedade de A
   e B serem iguais
9
10 #define pos(x) (2 * (x))
11 #define neg(x) (2 * (x) + 1)
12
13 int ord, idx, counter, vis[2 * MAX], inStack[2 * MAX], cycle[2 * MAX], scc[2 *
   MAX];
14 bool ans[MAX];
15 stack<int> stk;
16
17 void tarjan(int u) {
18     vis[u] = cycle[u] = ord++;
19     stk.push(u);
20     inStack[u] = 1;
21     for(int &v : graph[u]) {
22         if(vis[v] == 0) tarjan(v);
23         if(inStack[v]) cycle[u] = min(cycle[u], cycle[v]);
24     }
25     if(vis[u] == cycle[u]) {
26         int w;
27         do {
28             w = stk.top();
29             stk.pop();
30             scc[w] = counter;

```

```

31         inStack[w] = 0;
32     } while(w != u);
33     counter++;
34 }
35 }
36
37 void initTarjan() {
38     ord = 1;
39     counter = 0;
40     memset(vis, 0, sizeof(vis));
41     memset(inStack, 0, sizeof(inStack));
42     for(int u = 0; u < 2 * n; u++) {
43         if(vis[u] == 0) tarjan(u);
44     }
45 }
46
47 void setClause(int u, int v, int b1, int b2) {
48     int u1, u2, v1, v2;
49     u1 = neg(u);
50     u2 = pos(u);
51     v1 = neg(v);
52     v2 = pos(v);
53     if(b1 == 0) swap(u1, u2);
54     if(b2 == 0) swap(v1, v2);
55     graph[u1].push_back(v2);
56     graph[v1].push_back(u2);
57 }
58
59 int twoSat() {
60     initTarjan();
61     for(int i = 0; i < 2 * n; i += 2) {
62         if(scc[i] == scc[i + 1]) {
63             return 0;
64         } else {
65             if(scc[i] < scc[i + 1]) ans[i / 2] = 1;
66             else ans[i / 2] = 0;
67         }
68     }
69     return 1;
70 }

```

5.2 Articulations

```

1 // Detecta os pontos de articulação do grafo não dirigido
2 // artic[u] = 1 significa que removendo o vértice u, aumenta-se o número de
  componentes conexos do grafo
3
4 int timer, vis[MAX], tin[MAX], low[MAX], artic[MAX];
5
6 void dfs(int u, int p) {
7     vis[u] = 1;
8     tin[u] = low[u] = timer++;
9     for(int v : graph[u]) {
10         if(v != p) {
11             if(vis[v]) {
12                 low[u] = min(low[u], tin[v]);
13             } else {
14                 dfs(v, u);

```

```

15         low[u] = min(low[u], low[v]);
16         if((p != -1 && low[v] >= tin[u]) ||
17            (p == -1 && low[v] > tin[u] && graph[u].size() > 1)) {
18             artic[u] = 1;
19         }
20     }
21 }
22 }
23 }
24
25 void findArticulations() {
26     timer = 0;
27     memset(vis, 0, sizeof(vis));
28     memset(tin, -1, sizeof(tin));
29     memset(low, -1, sizeof(low));
30     memset(artic, 0, sizeof(artic));
31     for(int i = 0; i < n; i++) {
32         if(vis[i] == 0) dfs(i, -1);
33     }
34 }

```

5.3 Bellman Ford

```

1 // retorno 1 significa ciclo de peso negativo
2
3 int dist[MAX];
4
5 int bellmanFord(int src) {
6     int flag = 0;
7     for(int i = 0; i < n; i++) dist[i] = INF;
8     dist[src] = 0;
9     for(int i = 0; i < n; i++) {
10         flag = 0;
11         for(int u = 0; u < n; u++) {
12             for(auto &[v, dv] : graph[u]) {
13                 if(dist[u] != INF && dist[v] > dist[u] + dv) {
14                     dist[v] = dist[u] + dv;
15                     flag = 1;
16                 }
17             }
18         }
19     }
20     return flag;
21 }

```

5.4 Bfs

```

1 int dist[MAX], parent[MAX];
2
3 void bfs(int s) {
4     queue<int> q;
5     fill(dist, dist + n, INF);
6     dist[s] = 0;
7     parent[s] = -1;
8     q.push(s);
9     while(!q.empty()) {
10         int u = q.front();
11         q.pop();

```

```

12     for(int &v : graph[u]) {
13         if(dist[v] == INF) {
14             dist[v] = dist[u] + 1;
15             parent[v] = u;
16             q.push(v);
17         }
18     }
19 }
20 }

```

5.5 Bfs Grid

```

1  int dist[MAXG][MAXG];
2  int ai[] = {-1, 1, 0, 0};
3  int aj[] = {0, 0, 1, -1};
4
5  void bfs(int si, int sj) {
6      queue<pii> q;
7      int ii, jj;
8      for(int i = 0; i < n; i++) {
9          for(int j = 0; j < m; j++) {
10             dist[i][j] = INF;
11         }
12     }
13     dist[si][sj] = 0;
14     q.emplace(si, sj);
15     while(!q.empty()) {
16         auto [i, j] = q.front();
17         q.pop();
18         for(int k = 0; k < 4; k++) {
19             ii = i + ai[k];
20             jj = j + aj[k];
21             if(ii >= 0 && ii < n && jj >= 0 && jj < m && dist[ii][jj] == INF) {
22                 dist[ii][jj] = dist[i][j] + 1;
23                 q.emplace(ii, jj);
24             }
25         }
26     }
27 }

```

5.6 Bridges

```

1  int ans, timer, vis[MAX], tin[MAX], low[MAX];
2
3  void dfs(int u, int p) {
4      vis[u] = 1;
5      tin[u] = low[u] = timer++;
6      for(int v : graph[u]) {
7          if(v != p) {
8              if (vis[v]) {
9                  low[u] = min(low[u], tin[v]);
10             } else {
11                 dfs(v, u);
12                 low[u] = min(low[u], low[v]);
13                 if(low[v] > tin[u]) {
14                     // Aresta (u <-> v) é uma ponte, processe-a aqui
15                 }
16             }
17         }
18     }
19 }

```



```

17     }
18 }
19 }
20
21 void findBridges() {
22     timer = ans = 0;
23     memset(vis, 0, sizeof(vis));
24     memset(tin, -1, sizeof(tin));
25     memset(low, -1, sizeof(low));
26     for(int i = 0; i < n; i++) {
27         if(vis[i] == 0) dfs(i, -1);
28     }
29 }

```

5.7 De Bruijn

```

1 // dado um alfabeto A e um tamanho n, constrói uma string tal que
2 // todas as strings possíveis de tamanho n com o alfabeto A
3 // aparecem exatamente uma vez como substring da string de saída
4 // Complexidade:  $O(k^n)$ 
5
6 unordered_set<string> seen;
7 vector<int> edges;
8
9 void dfs(string node, int& k, string& A){
10     for(int i = 0; i < k; ++i) {
11         string str = node + A[i];
12         if(seen.find(str) == seen.end()) {
13             seen.insert(str);
14             dfs(str.substr(1), k, A);
15             edges.push_back(i);
16         }
17     }
18 }
19
20 string deBruijn(int n) {
21     string A = "01"; // alfabeto
22     int k = 2; // tamanho do alfabeto
23     // n = tamanho das substrings
24     seen.clear();
25     edges.clear();
26     string startingNode = string(n - 1, A[0]);
27     dfs(startingNode, k, A);
28     string S;
29     int l = pow(k, n);
30     for(int i = 0; i < l; ++i)
31         S += A[edges[i]];
32     S += startingNode;
33     return S;
34 }

```

5.8 Digraph Cycle

```

1 // retorna 1 se o grafo dirigido contém ao menos um ciclo
2
3 int vis[MAX];
4
5 int dfs(int u) {
6     vis[u] = 1;

```

```

7     for(int &v : graph[u]) {
8         if(vis[v] == 1) return 1;
9         if(vis[v] == 0 && dfs(v)) return 1;
10    }
11    vis[u] = 2;
12    return 0;
13 }
14
15 int findCycle() {
16     for(int i = 0; i < n; i++) {
17         if(vis[i] == 2) continue;
18         if(dfs(i)) return 1;
19     }
20     return 0;
21 }

```

5.9 Dijkstra

```

1  int dist[MAX], parent[MAX];
2
3  void dijkstra(int s) {
4      priority_queue<pii, vector<pii>, greater<pii>> pq;
5      fill(dist, dist + n, INF);
6      dist[s] = 0;
7      parent[s] = -1;
8      pq.emplace(0, s);
9      while(!pq.empty()) {
10         auto [du, u] = pq.top();
11         pq.pop();
12         if(du > dist[u]) continue;
13         for(auto &[v, dv] : graph[u]) {
14             if(dist[v] > du + dv) {
15                 dist[v] = du + dv;
16                 parent[v] = u;
17                 pq.emplace(du + dv, v);
18             }
19         }
20     }
21 }

```

5.10 Dinic Max Flow

```

1  struct FlowEdge {
2      int v, u;
3      ll cap, flow = 0;
4      FlowEdge(int v, int u, ll cap) : v(v), u(u), cap(cap) {}
5  };
6
7  struct Dinic {
8      const ll flow_inf = 1e18;
9      vector<FlowEdge> edges;
10     vector<vector<int>> adj;
11     int n, m = 0;
12     int s, t;
13     vector<int> level, ptr;
14     queue<int> q;
15
16     Dinic(int n, int s, int t) : n(n), s(s), t(t) {
17         adj.resize(n);

```

```

18     level.resize(n);
19     ptr.resize(n);
20 }
21
22 void add_edge(int v, int u, ll cap) {
23     edges.emplace_back(v, u, cap);
24     edges.emplace_back(u, v, 0);
25     adj[v].push_back(m);
26     adj[u].push_back(m + 1);
27     m += 2;
28 }
29
30 bool bfs() {
31     while(!q.empty()) {
32         int v = q.front();
33         q.pop();
34         for(int id : adj[v]) {
35             if(edges[id].cap == edges[id].flow)
36                 continue;
37             if(level[edges[id].u] != -1)
38                 continue;
39             level[edges[id].u] = level[v] + 1;
40             q.push(edges[id].u);
41         }
42     }
43     return level[t] != -1;
44 }
45
46 ll dfs(int v, ll pushed) {
47     if(pushed == 0) return 0;
48     if(v == t) return pushed;
49     for(int& cid = ptr[v]; cid < (int)adj[v].size(); cid++) {
50         int id = adj[v][cid];
51         int u = edges[id].u;
52         if(level[v] + 1 != level[u])
53             continue;
54         ll tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
55         if(tr == 0) continue;
56         edges[id].flow += tr;
57         edges[id ^ 1].flow -= tr;
58         return tr;
59     }
60     return 0;
61 }
62
63 ll flow() {
64     ll f = 0;
65     while(true) {
66         fill(level.begin(), level.end(), -1);
67         level[s] = 0;
68         q.push(s);
69         if(!bfs()) break;
70         fill(ptr.begin(), ptr.end(), 0);
71         while(ll pushed = dfs(s, flow_inf)) {
72             f += pushed;
73         }
74     }

```

```

75         return f;
76     }
77 };

```

5.11 Dsu

```

1  int parent[MAX], nodeRank[MAX];
2
3  int findSet(int u) {
4      if(parent[u] == u) return u;
5      return parent[u] = findSet(parent[u]);
6  }
7
8  void unionSets(int u, int v) {
9      u = findSet(u);
10     v = findSet(v);
11     if(nodeRank[u] > nodeRank[v]) {
12         parent[v] = u;
13     } else {
14         parent[u] = v;
15         if(nodeRank[v] == nodeRank[u]) nodeRank[v]++;
16     }
17 }
18
19 void preprocess() {
20     fill(nodeRank, nodeRank + n, 0);
21     iota(parent, parent + n, 0);
22 }

```

5.12 Erdos Gallai

```

1  // Teorema de Erdős-Gallai
2  // Diz se uma lista contendo graus de vértices de um grafo não dirigido
3  // pode de fato compor um grafo válido
4  // Retorno = 1 é válido, 0 = não válido
5  // Os graus devem estar em deg, 1-indexed
6
7  int deg[MAX], rng[MAX];
8  ll l[MAX];
9
10 int erdosGallai() {
11     int x, flag;
12     ll total = 0;
13     for(int i = 1; i <= n; i++) {
14         total += deg[i];
15     }
16     if((total & 1) == 0) {
17         flag = 1;
18         sort(deg + 1, deg + n + 1, greater<int>());
19         l[0] = 0;
20         for(int i = 1; i <= n; i++) {
21             l[i] = deg[i] + l[i - 1];
22         }
23         l[n + 1] = l[n];
24         x = n;
25         for(int i = 1; i <= n; i++) {
26             while(deg[x] < i) x--;
27             rng[i] = max(x, i);
28         }

```

```

29         for(int k = 1; k <= n; k++) {
30             ll y = (ll) (rng[k] - k) * k + l[n] - l[rng[k]];
31             if(l[k] > (ll) k * (k - 1) + y) {
32                 flag = 0;
33                 break;
34             }
35         }
36     } else flag = 0;
37     return flag;
38 }

```

5.13 Euler Path Directed

```

1 // euler path para grafos dirigidos
2 // aqui, inicio = 0 e final = n - 1
3 // m deve conter o número de arestas
4
5 int idx, path[MAX], inDegree[MAX], outDegree[MAX];
6 stack<int> stk;
7
8 void eulerPath(int start) {
9     stk.push(start);
10    idx = 0;
11    while(!stk.empty()) {
12        int u = stk.top();
13        if(outDegree[u] == 0) {
14            path[idx++] = u;
15            stk.pop();
16        } else {
17            int v = graph[u].back();
18            graph[u].pop_back();
19            outDegree[u]--;
20            stk.push(v);
21        }
22    }
23    reverse(path, path + idx);
24 }
25
26 void setEdge(int u, int v) {
27     graph[u].push_back(v);
28     outDegree[u]++;
29     inDegree[v]++;
30 }
31
32 bool isValid() {
33     int flag = 1, start = 0, ending = n - 1;
34     if(outDegree[start] - inDegree[start] != 1 || inDegree[ending] - outDegree[
35         ending] != 1) {
36         flag = 0;
37     } else {
38         for(int i = 1; i < ending; i++) {
39             if(inDegree[i] != outDegree[i]) {
40                 flag = 0;
41                 break;
42             }
43         }
44     }
45     if(flag) {

```

```

45     eulerPath(start);
46     if(idx == m + 1 && path[idx - 1] == n - 1) {
47         for(int i = 0; i < idx; i++) {
48             cout << path[i] + 1 << ' ';
49         }
50         cout << '\n';
51     } else flag = 0;
52 }
53 return flag;
54 }

```

5.14 Euler Tour

```

1  int idx, counter, tot, ans[MAX], degree[MAX], vis[MAX];
2  vector<unordered_set<int>> graph(MAX);
3  stack<int> stk;
4
5  void eulerTour() {
6      int u, v;
7      idx = 0;
8      stk.push(0); // origem do tour
9      while(!stk.empty()) {
10         u = stk.top();
11         if(graph[u].empty()) {
12             ans[idx++] = u;
13             stk.pop();
14         } else {
15             v = *(graph[u].begin());
16             stk.push(v);
17             graph[u].erase(v);
18             graph[v].erase(u);
19         }
20     }
21 }
22
23 void dfs(int u) {
24     vis[u] = 1;
25     counter += graph[u].size();
26     for(int v : graph[u]) {
27         if(vis[v] == 0) dfs(v);
28     }
29 }
30
31 bool isValid() {
32     for(int i = 0; i < n; i++) {
33         if(degree[i] & 1) return 0;
34     }
35     dfs(0);
36     if(counter != tot) return 0;
37     eulerTour();
38     return 1;
39 }
40
41 void setEdge(int u, int v) {
42     degree[u]++;
43     degree[v]++;
44     tot += 2;
45     graph[u].insert(v);

```

```

46     graph[v].insert(u);
47 }

```

5.15 Ffek Max Flow

```

1  // Ford-Fulkerson-Edmonds-Karp
2  //  $O(V * E^2)$ 
3
4  int parent[MAX], dist[MAX], cap[MAX][MAX];
5
6  int bfs(int s, int t) {
7      int u, bneck = INF;
8      queue<int> q;
9      for(u = 0; u < n; u++) dist[u] = INF;
10     dist[s] = 0;
11     q.push(s);
12     while(!q.empty()) {
13         u = q.front();
14         q.pop();
15         if(u == t) break;
16         for(int &v : graph[u]) {
17             if(cap[u][v] > 0 && dist[v] == INF) {
18                 dist[v] = dist[u] + 1;
19                 parent[v] = u;
20                 q.push(v);
21             }
22         }
23     }
24     if(dist[t] == INF) return 0;
25     for(u = t; u != s; u = parent[u]) {
26         bneck = bneck > cap[parent[u]][u] ? cap[parent[u]][u] : bneck;
27     }
28     return bneck;
29 }
30
31 int ffek(int s, int t) {
32     int u, bneck, f = 0;
33     while((bneck = bfs(s, t))) {
34         for(u = t; u != s; u = parent[u]) {
35             cap[parent[u]][u] -= bneck;
36             cap[u][parent[u]] += bneck;
37         }
38         f += bneck;
39     }
40     return f;
41 }
42
43 void setEdge(int u, int v, int w) {
44     if(cap[u][v] == 0) {
45         graph[u].push_back(v);
46         graph[v].push_back(u);
47     }
48     cap[u][v] += w;
49     // se a aresta for bidirecional, adicionar w ao cap[v][u] também
50 }

```

5.16 Floyd Warshall Path

```

1  int sz, path[MAXG], graph[MAXG][MAXG], parent[MAXG][MAXG];
2
3  void floydWarshall() {
4      for(int k = 0; k < n; k++) {
5          for(int i = 0; i < n; i++) {
6              for(int j = 0; j < n; j++) {
7                  if(graph[i][j] > graph[i][k] + graph[k][j]) {
8                      graph[i][j] = graph[i][k] + graph[k][j];
9                      parent[i][j] = parent[k][j];
10                 }
11             }
12         }
13     }
14 }
15
16 void recover(int u, int v) {
17     while(u != v) {
18         path[sz++] = v;
19         v = parent[u][v];
20     }
21     path[sz++] = u;
22 }
23
24 void setEdge(int u, int v) {
25     graph[u][v] = 1;
26     graph[v][u] = 1;
27     parent[u][v] = u;
28     parent[v][u] = v;
29 }

```

5.17 Held Karp

```

1  // calcula o ciclo hamiltoniano de menor custo
2
3  // atenção, o vértice de origem aqui é o 0! fortes ajustes devem ser feitos
4  // caso a origem desejada seja outro vértice
5
6  int graph[MAX_V][MAX_V], dp[1 << MAX_V][MAX_V];
7
8  int heldKarp() {
9      int i, j, ans, mask, last, next, newMask;
10     for(mask = 0; mask < (1 << n); mask++) {
11         for(i = 0; i < n; i++) {
12             dp[mask][i] = INF;
13         }
14     }
15     dp[1][0] = 0;
16     for(mask = 1; mask < (1 << n); mask++) {
17         for(last = 0; last < n; last++) {
18             if(dp[mask][last] == INF) continue;
19             for(next = 0; next < n; next++) {
20                 if(!(mask & (1 << next))) {
21                     newMask = mask | (1 << next);
22                     dp[newMask][next] = min(dp[newMask][next], dp[mask][last] +
23                                             graph[last][next]);
24                 }
25             }
26         }
27     }
28 }

```



```

26     }
27     ans = INF;
28     for (i = 1; i < n; i++) {
29         ans = min(ans, dp[(1 << n) - 1][i]); // + graph[i][0] caso o retorno ao
           início seja necessário. i = último vértice escolhido do caminho e 0 é
           o vértice de início
30     }
31     return ans;
32 }

```

5.18 Hungarian Min Cost Matching

```

1  ll w[MAX][MAX], mat[MAX][MAX], pi[MAX], pj[MAX], p[MAX], way[MAX], ans[MAX];
2  //pi e ans = n + 1, o resto m + 1;
3
4  //É possível transformar o problema de minimização em um de maximização,
5  //basta multiplicar todos os valores de mat[][] por -1 e printar -hungarian()
6  //TUDO ESTÁ INDEXADO EM 1!!!!
7
8  ll hungarian() {
9      for(int i = 1; i <= n; i++) {
10         p[0] = i;
11         int j0 = 0;
12         vector<ll> minv(m + 1, INFL);
13         vector<bool> used(m + 1, false);
14         do {
15             used[j0] = true;
16             ll i0 = p[j0], delta = INFL, j1;
17             for(int j = 1; j <= m; j++) {
18                 if(!used[j]) {
19                     ll cur = mat[i0][j] - pi[i0] - pj[j];
20                     if(cur < minv[j]) minv[j] = cur, way[j] = j0;
21                     if(minv[j] < delta) delta = minv[j], j1 = j;
22                 }
23             }
24             for(int j = 0; j <= m; j++) {
25                 if(used[j]) pi[p[j]] += delta, pj[j] -= delta;
26                 else minv[j] -= delta;
27             }
28             j0 = j1;
29         } while(p[j0] != 0);
30         do {
31             int j1 = way[j0];
32             p[j0] = p[j1];
33             j0 = j1;
34         } while(j0);
35     }
36     for(int j = 1; j <= m; j++) ans[p[j]] = j; //recupera a resposta
37     return -pj[0];
38 }
39
40 int main() {
41     ios::sync_with_stdio(0);
42     cin.tie(0);
43     int u, v;
44     ll cost;
45     cin >> n;
46     for(int i = 0; i < n; i++) {

```

```

47     for(int j = 0; j < n; j++) {
48         cin >> w[i][j];
49     }
50 }
51 for(int i = 0; i < n; i += 2) {
52     for(int j = 0; j < n; j += 2) {
53         mat[i / 2 + 1][j / 2 + 1] = (j - 1 >= 0 ? w[j - 1][i] : 0) + (j + 1
            < n ? w[i][j + 1] : 0);
54     }
55 }
56 n = (n + 1) / 2;
57 m = n;
58 cout << hungarian() << '\n';
59 return 0;
60 }

```

5.19 Kruskal

```

1  int parent[MAX], nodeRank[MAX];
2  vector<tiii> edges;
3
4  int findSet(int u) {
5      if(parent[u] == u) return u;
6      return parent[u] = findSet(parent[u]);
7  }
8
9  void unionSets(int u, int v) {
10     u = findSet(u);
11     v = findSet(v);
12     if(nodeRank[u] > nodeRank[v]) {
13         parent[v] = u;
14     } else {
15         parent[u] = v;
16         if(nodeRank[v] == nodeRank[u]) nodeRank[v]++;
17     }
18 }
19
20 int kruskal() {
21     int ans = 0, t = n;
22     fill(nodeRank, nodeRank + n, 0);
23     iota(parent, parent + n, 0);
24     sort(edges.begin(), edges.end());
25     for(auto &[u, v, w] : edges) {
26         if(findSet(u) != findSet(v)) {
27             ans += w;
28             unionSets(u, v);
29             t--;
30         }
31     }
32     return t == 1 ? ans : -1;
33 }

```

5.20 Max Matching Aug

```

1  // na variável n deve estar o tamanho do conjunto dos vértices
2  // da "esquerda" do grafo bipartido
3
4  // as arestas do grafo são direcionadas, somente inserir arestas da esquerda
   para a direita

```

```

5
6
7 int vis[MAX], match[MAX];
8
9 int dfs(int u) {
10     vis[u] = 1;
11     for(int &v : graph[u]) {
12         if(match[v] == -1 || (vis[match[v]] == 0 && dfs(match[v]))) {
13             match[v] = u;
14             return 1;
15         }
16     }
17     return 0;
18 }
19
20 int aug() {
21     memset(match, -1, sizeof(match));
22     int ans = 0;
23     for(int u = 0; u < n; u++) {
24         memset(vis, 0, sizeof(vis));
25         if(vis[u] == 0) ans += dfs(u);
26     }
27     return ans;
28 }

```

5.21 Max Matching Hopcroft Karp

```

1 // na variável n deve estar o tamanho do conjunto dos vértices
2 // da "esquerda" do grafo bipartido
3
4 // as arestas do grafo são direcionadas, somente inserir arestas da esquerda
   para a direita
5
6 int match[MAX], vis[MAX], dist[MAX];
7
8 int dfs(int u) {
9     int w;
10    vis[u] = 1;
11    for(int &v : graph[u]) {
12        w = match[v];
13        if(w == -1 || (dist[w] == dist[u] + 1 && vis[w] == 0 && dfs(w))) {
14            match[v] = u;
15            match[u] = v;
16            return 1;
17        }
18    }
19    return 0;
20 }
21
22 int bfs() {
23     int u, w, i;
24     queue<int> q;
25     for(u = 0; u < n; u++) {
26         if(match[u] == -1) {
27             dist[u] = 0;
28             q.push(u);
29         } else dist[u] = INF;
30     }
31     while(!q.empty()) {

```

```

32     u = q.front();
33     q.pop();
34     for(int &v : graph[u]) {
35         w = match[v];
36         if(w == -1) return 1;
37         if(dist[w] == INF) {
38             dist[w] = dist[u] + 1;
39             q.push(w);
40         }
41     }
42 }
43 return 0;
44 }
45
46 int hopcroftKarp() {
47     int u, ans = 0;
48     memset(match, -1, sizeof(match));
49     while(bfs()) {
50         memset(vis, 0, sizeof(vis));
51         for(u = 0; u < n; u++) {
52             if(match[u] == -1 && vis[u] == 0) ans += dfs(u);
53         }
54     }
55     return ans;
56 }

```

5.22 Min Cost Max Flow Dijkstra

```

1  typedef pair<ll, ll> pll;
2
3  int start[MAX], parent[MAX];
4  ll d, k, prio[MAX], pot[MAX];
5  vector<edge> eg;
6
7  struct edge {
8      int u, v, nxt; ll cap, cost;
9      edge() {}
10     edge(int _u, int _v, int _nxt, ll _cap, ll _cost) :
11         u(_u), v(_v), nxt(_nxt), cap(_cap), cost(_cost) {}
12 };
13
14 ll dijkstra(int s, int t) {
15     // retorna o bneck, ou 0 se nenhum caminho foi encontrado
16     priority_queue<pll, vector<pll>, greater<pll>> pq;
17     fill(prio, prio + n, INFLL);
18     memset(parent, -1, sizeof(parent));
19     prio[s] = 0;
20     pq.emplace(0, s);
21     while(!pq.empty()) {
22         auto [du, u] = pq.top();
23         pq.pop();
24         if(prio[u] < du) continue;
25         for(int i = start[u]; i != -1; i = eg[i].nxt) {
26             int v = eg[i].v;
27             if(eg[i].cap > 0 && prio[v] > du + eg[i].cost + pot[u] - pot[v]) {
28                 prio[v] = du + eg[i].cost + pot[u] - pot[v];
29                 pq.emplace(prio[v], v);
30                 parent[v] = i;

```

```

31     }
32 }
33 }
34 if(prio[t] == INFLL) return 0;
35 ll bneck = INFLL;
36 for(int i = parent[t]; i != -1; i = parent[eg[i].u]) {
37     bneck = min(bneck, eg[i].cap);
38 }
39 return bneck;
40 }
41
42 void bellmanFord(int s) {
43     fill(pot, pot + n, INFLL);
44     pot[s] = 0;
45     for(int i = 0; i < n; i++) {
46         for(int j = 0; j < n; j++) {
47             for(int k = start[j]; k != -1; k = eg[k].nxt) {
48                 int v = eg[i].v;
49                 if(eg[i].cap > 0 && pot[v] > pot[j] + eg[i].cost) {
50                     pot[v] = pot[j] + eg[i].cost;
51                 }
52             }
53         }
54     }
55 }
56
57 pll minCostMaxFlow(int s, int t) {
58     ll f = 0, cost = 0, bneck;
59     bellmanFord(s);
60     while(bneck = dijkstra(s, t)) {
61         f += bneck;
62         for(int i = 0; i < n; i++) pot[i] += prio[i];
63         for(int i = parent[t]; i != -1; i = parent[eg[i].u]) {
64             eg[i].cap -= bneck;
65             eg[i ^ 1].cap += bneck;
66             cost += eg[i].cost * bneck;
67         }
68     }
69     return {f, cost};
70 }
71
72 void addEdge(int u, int v, ll cap, ll cost) {
73     eg.emplace_back(u, v, start[u], cap, cost);
74     start[u] = eg.size() - 1;
75 }
76
77 void setGraph() {
78     int u, v, c = 1;
79     ll cost;
80     eg.clear();
81     memset(start, -1, sizeof(start));
82     for(int i = 0; i < m; i++) {
83         cin >> u >> v >> cost;
84         addEdge(u, v, 1, cost); //aresta original
85         addEdge(v, u, 0, -cost); //aresta reversa, deve ser imediatamente a
86         //se o grafo for não direcionado, inserir mais as outras duas arestas

```

```

        abaixo
87     addEdge(v, u, 1, cost);
88     addEdge(u, v, 0, -cost);
89 }
90 cin >> d >> k;
91 for(auto &e : eg) e.cap *= k;
92 addEdge(0, 1, d, 0);
93 addEdge(1, 0, 0, 0);
94 n++;
95 cin >> d >> k;
96 for(auto &e : eg) e.cap *= k;
97 addEdge(0, 1, d, 0);
98 addEdge(1, 0, 0, 0);
99 n++;
100 //neste problema, é preciso saber o custo mínimo de se enviar
101 // 'd' unidades de fluxo, que não necessariamente é máximo.
102 //portanto, um vértice extra, o zero, é colocado no grafo, com uma aresta
103 //de capacidade 'd' indo para o vértice 1, para limitar o fluxo em 'd'.
104 //os outros vértices, que normalmente teriam seus identificadores
    decrementados
105 //para trabalhar com indexação em 0, permanecem os mesmos. finalmente, n é
    incrementado
106 //para corresponder ao novo número de vértices do grafo.
107 pll ans = minCostMaxFlow(0, n - 1);
108 cout << "Instancia " << c << '\n';
109 if(ans.first < d) cout << "impossivel\n\n";
110 else cout << ans.second << "\n\n";
111 c++;
112 }

```

5.23 Min Cost Max Flow Spfa

```

1 //UVA 10594
2
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 #define MAX 112
7 #define MAXG 1123
8 #define INFL 11234567891234567
9 #define MOD 998244353
10 #define EPS 1e-9
11 typedef long long ll;
12 typedef pair<int, int> pii;
13 typedef pair<ll, ll> pll;
14 typedef tuple<int, int, int> tiii;
15
16 struct edge {
17     int u, v, nxt; ll cap, cost;
18     edge() {}
19     edge(int _u, int _v, int _nxt, ll _cap, ll _cost) :
20         u(_u), v(_v), nxt(_nxt), cap(_cap), cost(_cost) {}
21 };
22
23 int n, m, arr[MAX], inQueue[MAX], start[MAX], parent[MAX];
24 ll d, k, dist[MAX];
25 vector<edge> eg;
26
27 ll spfa(int s, int t) {

```

```

28 // retorna o bneck, ou 0 se nenhum caminho foi encontrado
29 fill(dist, dist + n, INFLL);
30 memset(inQueue, 0, sizeof(inQueue));
31 memset(parent, -1, sizeof(parent));
32 queue<int> q;
33 dist[s] = 0;
34 inQueue[s] = 1;
35 q.push(s);
36 while(!q.empty()) {
37     int u = q.front();
38     q.pop();
39     inQueue[u] = 0;
40     for(int i = start[u]; i != -1; i = eg[i].nxt) {
41         int v = eg[i].v;
42         if(eg[i].cap > 0 && dist[v] > dist[u] + eg[i].cost) {
43             dist[v] = dist[u] + eg[i].cost;
44             parent[v] = i;
45             if(inQueue[v] == 0) {
46                 q.push(v);
47                 inQueue[v] = 1;
48             }
49         }
50     }
51 }
52 if(dist[t] == INFLL) return 0;
53 ll bneck = INFLL;
54 for(int i = parent[t]; i != -1; i = parent[eg[i].u]) {
55     bneck = min(bneck, eg[i].cap);
56 }
57 return bneck;
58 }
59
60 pll minCostMaxFlowFfek(int s, int t) {
61     ll f = 0, cost = 0, bneck;
62     while(bneck = spfa(s, t)) {
63         f += bneck;
64         cost += bneck * dist[t];
65         for(int i = parent[t]; i != -1; i = parent[eg[i].u]) {
66             eg[i].cap -= bneck;
67             eg[i ^ 1].cap += bneck;
68         }
69     }
70     return {f, cost};
71 }
72
73 void addEdge(int u, int v, ll cap, ll cost) {
74     eg.emplace_back(u, v, start[u], cap, cost);
75     start[u] = eg.size() - 1;
76 }
77
78 int main() {
79     ios::sync_with_stdio(0);
80     cin.tie(0);
81     int u, v, c = 1;
82     ll cost;
83     while(cin >> n >> m) {
84         eg.clear();

```

```

85     memset(start, -1, sizeof(start));
86     for(int i = 0; i < m; i++) {
87         cin >> u >> v >> cost;
88         addEdge(u, v, 1, cost); //aresta original
89         addEdge(v, u, 0, -cost); //aresta reversa, deve ser imediatamente a
           proxima a ser inserida
90         addEdge(v, u, 1, cost); //o grafo do problema é não direcionado!
           portanto, as outras duas arestas devem ser inseridas
91         addEdge(u, v, 0, -cost);
92     }
93     cin >> d >> k;
94     for(auto &e : eg) e.cap *= k;
95     addEdge(0, 1, d, 0);
96     addEdge(1, 0, 0, 0);
97     n++;
98     //neste problema, é preciso saber o custo mínimo de se enviar
99     // 'd' unidades de fluxo, que não necessariamente é máximo.
100    //portanto, um vértice extra, o zero, é colocado no grafo, com uma
           aresta
101    //de capacidade 'd' indo para o vértice 1, para limitar o fluxo em 'd'.
102    //os outros vértices, que normalmente teriam seus identificadores
           decrementados
103    //para trabalhar com indexação em 0, permanecem os mesmos. finalmente, n
           é incrementado
104    //para corresponder ao novo número de vértices do grafo.
105    pll ans = minCostMaxFlowFfek(0, n - 1);
106    cout << "Instancia " << c << '\n';
107    if(ans.first < d) cout << "impossivel\n\n";
108    else cout << ans.second << "\n\n";
109    c++;
110 }
111 return 0;
112 }

```

5.24 Prim

```

1  int vis[MAX];
2
3  int prim(int s) {
4      priority_queue<pii, vector<pii>, greater<pii>> pq;
5      int ans = 0, t = 0;
6      pq.emplace(0, s);
7      while(!pq.empty()) {
8          auto [du, u] = pq.top();
9          pq.pop();
10         if(vis[u]) continue;
11         vis[u] = 1;
12         t++;
13         ans += du;
14         for(auto &[v, dv] : graph[u]) {
15             if(vis[v] == 0) pq.emplace(dv, v);
16         }
17     }
18     return t == n ? ans : -1;
19 }

```

5.25 Scc Dag

```

1  // constrói o grafo de condensação dos componentes fortemente conectados

```



```

2 // o grafo resultante será um DAG (directed acyclic graph)
3
4 // deve ser utilizado após rodar o algoritmo de Tarjan
5
6 vector<set<int>> dag(MAX);
7
8 void setDag() {
9     for(int u = 0; u < n; u++) {
10         for(int &v : graph[u]) {
11             if(scc[u] != scc[v]) {
12                 dag[scc[u]].insert(scc[v]);
13             }
14         }
15     }
16 }

```

5.26 Spfa

```

1 // Shortest Path Faster Algorithm (Bellman-Ford que funciona melhorzinho na
  // prática)
2 // Complexidade continua sendo  $O(v^3)$ 
3
4 // retorno 1 significa contém ciclo de peso negativo
5
6 int cnt[MAX], inQueue[MAX], dist[MAX];
7
8 int spfa() {
9     fill(dist, dist + n, INFL);
10    queue<int> q;
11    dist[0] = 0;
12    inQueue[0] = 1;
13    q.push(0);
14    while(!q.empty()) {
15        int u = q.front();
16        q.pop();
17        inQueue[u] = 0;
18        for(auto &[v, dv] : graph[u]) {
19            if(dist[v] > dist[u] + dv) {
20                dist[v] = dist[u] + dv;
21                if(inQueue[v] == 0) {
22                    q.push(v);
23                    inQueue[v] = 1;
24                    cnt[v]++;
25                    if(cnt[v] > n) return 1;
26                }
27            }
28        }
29    }
30    return 0;
31 }

```

5.27 Sucessor Graph

```

1 //Atenção: se a lista de nxt é uma permutação, o grafo conterá apenas ciclos!
2 //não haverão vértices de fora que levam a um ciclo. isso facilita muito
3 //o processamento do grafo
4
5 int nxt[MAX], comp[MAX], cyc[MAX], pos[MAX], dist[MAX], vis[MAX], up[MAX][LOG];
6

```

```

7 void dfs(int u, int c) {
8     vis[u] = 1;
9     comp[u] = c;
10    for(int &v : graph[u]) {
11        if(vis[v] == 0) dfs(v, c);
12    }
13 }
14
15 void binaryLifting() {
16     for(int i = 0; i < n; i++) {
17         up[i][0] = nxt[i];
18     }
19     for(int i = 1; i < LOG; i++) {
20         for(int j = 0; j < n; j++) {
21             up[j][i] = up[up[j][i - 1]][i - 1];
22         }
23     }
24 }
25
26 void findCycles(int u) {
27     vis[u] = 1;
28     if(vis[nxt[u]] == 1) {
29         if(cyc[nxt[u]] == -1) {
30             int v = u;
31             int p = 0;
32             do {
33                 pos[v] = p++;
34                 dist[v] = 0;
35                 v = nxt[v];
36             } while(v != u);
37             do {
38                 cyc[v] = p;
39                 v = nxt[v];
40             } while(v != u);
41         } else {
42             dist[u] = dist[nxt[u]] + 1;
43             cyc[u] = 0;
44         }
45     } else {
46         findCycles(nxt[u]);
47         if(cyc[u] == -1) {
48             dist[u] = dist[nxt[u]] + 1;
49             cyc[u] = 0;
50         }
51     }
52 }
53
54 int getCycleDist(int u, int v) {
55     int a = pos[u], b = pos[v];
56     if(b < a) swap(a, b);
57     return min(b - a, cyc[u] - b + a);
58 }
59
60 int minDistMeetingVertex(int u, int v) {
61     int a, b, d, ans;
62     if(comp[u] != comp[v]) ans = -1;
63     else if(u == v) ans = 0;

```

```

64     else {
65         if(cyc[v] != 0) swap(u, v);
66         if(cyc[u] != 0 && cyc[v] != 0) {
67             ans = getCycleDist(u, v);
68         } else if(cyc[u] != 0 && cyc[v] == 0) {
69             d = dist[v];
70             for(int i = 0; i < LOG; i++) {
71                 if(d & (1 << i)) v = up[v][i];
72             }
73             ans = d;
74             if(u != v) {
75                 ans += getCycleDist(u, v);
76             }
77         } else {
78             if(dist[v] < dist[u]) swap(u, v);
79             a = dist[v] - dist[u];
80             for(int i = 0; i < LOG; i++) {
81                 if(a & (1 << i)) {
82                     v = up[v][i];
83                 }
84             }
85             ans = a;
86             if(u != v) {
87                 b = dist[u];
88                 ans += 2 * b;
89                 int u1 = u, v1 = v;
90                 for(int i = LOG - 1; i >= 0; i--) {
91                     if(b & (1 << i)) {
92                         u1 = up[u1][i];
93                         v1 = up[v1][i];
94                     }
95                 }
96                 if(u1 == v1) {
97                     for(int i = LOG - 1; i >= 0; i--) {
98                         if(up[u][i] != up[v][i]) {
99                             u = up[u][i];
100                             v = up[v][i];
101                         }
102                     }
103                     u = nxt[u];
104                     v = nxt[v];
105                     ans -= 2 * dist[u];
106                 } else {
107                     ans += getCycleDist(u1, v1);
108                 }
109             }
110         }
111     }
112     return ans;
113 }
114
115 void readAndProcess() {
116     int u, c = 0;
117     for(int i = 0; i < n; i++) {
118         cin >> u;
119         u--;
120         nxt[i] = u;

```

```

121     graph[i].push_back(u);
122     graph[u].push_back(i);
123 }
124 fill(vis, vis + n, 0);
125 for(int i = 0; i < n; i++) {
126     if(vis[i] == 0) {
127         dfs(i, c);
128         c++;
129     }
130 }
131 fill(cyc, cyc + n, -1);
132 fill(vis, vis + n, 0);
133 for(int i = 0; i < n; i++) {
134     if(vis[i] == 0) findCycles(i);
135 }
136 binaryLifting();
137 }

```

5.28 Tarjan

```

1  int ord, counter, vis[MAX], scc[MAX], cycle[MAX], inStack[MAX];
2  stack<int> stk;
3
4  void tarjan(int u) {
5      vis[u] = cycle[u] = ord++;
6      stk.push(u);
7      inStack[u] = 1;
8      for(int &v : graph[u]) {
9          if(vis[v] == 0) tarjan(v);
10         if(inStack[v]) cycle[u] = min(cycle[u], cycle[v]);
11     }
12     if(vis[u] == cycle[u]) {
13         int w;
14         do {
15             w = stk.top();
16             stk.pop();
17             scc[w] = counter;
18             inStack[w] = 0;
19         } while(w != u);
20         counter++;
21     }
22 }
23
24 void initTarjan() {
25     ord = 1;
26     counter = 0;
27     memset(vis, 0, sizeof(vis));
28     memset(inStack, 0, sizeof(inStack));
29     for(int u = 0; u < n; u++) {
30         if(vis[u] == 0) tarjan(u);
31     }
32 }

```

6 Math

6.1 Binary Matrix Power

```

1 // Exponenciação de matrizes binárias, onde os valores são somente 0 ou 1

```

```

2 // Utiliza o "método dos quatro russos" para otimizar a parte de multiplicação
3
4 // Valores iniciais e resultado ficam em 'a'
5
6 vector<bitset<MAXG>> a, b;
7
8 void multmat(vector<bitset<MAXG>> &a, vector<bitset<MAXG>> &b, int ord) {
9     vector<bitset<MAXG>> result(ord);
10    for(int i = 0; i < ord; i++) {
11        result[i].reset();
12        for(int k = 0; k < ord; k++) {
13            if(a[i][k]) {
14                result[i] |= b[k];
15            }
16        }
17    }
18    a = result;
19 }
20
21 void ident(vector<bitset<MAXG>> &a, int ord) {
22     a.resize(ord);
23     for(int i = 0; i < ord; i++) {
24         a[i].reset();
25         a[i][i] = 1;
26     }
27 }
28
29 void expbin(vector<bitset<MAXG>> &a, int k, int ord) {
30     ident(b, ord);
31     while(k) {
32         if(k & 1) {
33             multmat(b, a, ord);
34         }
35         multmat(a, a, ord);
36         k >>= 1;
37     }
38     a = b;
39 }

```

6.2 Binomial Coefficient Nomod

```

1 //calcula os coeficientes binomiais para qualquer número n de 1 até cn, para um
  ck fixo
2 //vai bem quando computar os fatoriais para a fórmula direta não é possível
3 //(quando falta um mod para as operações, por exemplo)
4
5
6 // forma fácil, mas cuidado com overflow
7 ll cn, ck, cf[MAX];
8
9 void compute() {
10     for(int i = 0; i < ck; i++) cf[i] = 0;
11     cf[ck] = 1;
12     for(ll i = ck + 1; i <= cn; i++) {
13         __int128 t = (__int128) cf[i - 1] * i;
14         t /= (i - ck);
15         cf[i] = (ll) t;
16     }

```

```
17 }
```

6.3 Digits

```
1 // Conta quantos dígitos existem em todos os números de 1 a n
2
3 ll totalDigits(ll n) {
4     ll lans = 0;
5     for(ll i = 1; i <= n; i *= 10LL) {
6         lans += (n - i + 1LL);
7     }
8     return lans;
9 }
10
11 // Computa a soma de todos os dígitos dos números de 1 a n
12
13 ll dp(ll n, vector<ll> &dg) {
14     if(n < 10) return (n * (n + 1LL) / 2LL);
15     int d = (int)(log10(n));
16     ll p = (ll)(ceil(pow(10, d)));
17     ll msd = n / p;
18     return (msd * dg[d] + (msd * (msd - 1LL) / 2LL) * p
19         + msd * (1LL + n % p) + dp(n % p, dg));
20 }
21
22 ll sumOfDigits(ll n) {
23     int d = max(((int)(log10(n))), 1);
24     vector<ll> dg(d + 1);
25     dg[0] = 0;
26     dg[1] = 45;
27     for(int i = 2; i <= d; i++) {
28         dg[i] = dg[i - 1] * 10LL + 45LL * (ll)(ceil(pow(10, i - 1)));
29     }
30     return dp(n, dg);
31 }
```

6.4 Expbin

```
1 ll expbin(ll a, ll b) {
2     if(b == 0) return 1;
3     if(b & 1) return a * expbin(a, b - 1) % MOD;
4     ll c = expbin(a, b >> 1);
5     return c * c % MOD;
6 }
7
8 ll expbin(ll a, ll b) {
9     ll ans = 1;
10    while(b) {
11        if(b & 1) ans = (ans * a) % MOD;
12        a = (a * a) % MOD;
13        b >>= 1;
14    }
15    return ans;
16 }
```

6.5 Extended Gcd

```
1 // calcula o inverso multiplicativo de um numero mod não primo, que pode não
   existir
```

```

2
3 ll extended_gcd(ll a, ll b, ll &x, ll &y) {
4     if (b == 0) {
5         x = 1;
6         y = 0;
7         return a;
8     }
9     ll x1, y1;
10    ll d = extended_gcd(b, a % b, x1, y1);
11    x = y1;
12    y = x1 - y1 * (a / b);
13    return d;
14 }
15
16 ll modInverse(ll a, ll mod) {
17     ll x, y;
18     ll g = extended_gcd(a, mod, x, y);
19     if (g != 1) {
20         // a and m aren't relatively prime, so modular inverse doesn't exist
21         return -1;
22     } else {
23         return (x % mod + mod) % mod;
24     }
25 }

```

6.6 Fibonacci

```

1 // fibonacci rápido e fácil
2 // retorna {f(n), f(n + 1)}
3
4 pll fib(ll n) {
5     if(n == 0) return {0, 1};
6     auto p = fib(n >> 1);
7     ll c = p.first * (2LL * p.second - p.first);
8     ll d = p.first * p.first + p.second * p.second;
9     if(n & 1) return {d, c + d};
10    else return {c, d};
11 }
12
13 // fibonacci com pisano period
14 // necessário utilizar em conjunto com o código de sieve.cpp
15
16 int gcd(int a, int b) {
17     if(b == 0) return a;
18     return gcd(b, a % b);
19 }
20
21 int calcPeriod(int p, int exp) {
22     int ans = pow(p, exp - 1);
23     if(p == 2) return ans * 3;
24     else if(p == 5) return ans * 20;
25     else {
26         int mod = p % 10;
27         if(mod == 1 || mod == 9) return ans * (p - 1);
28         else return ans * 2 * (p + 1);
29     }
30     return 0;
31 }

```

```

32
33 int findPeriod(int n) {
34     int exp, mod, aux, lcm = 1;
35     for(int i = 0; i < pi && primes[i] * primes[i] <= n; i++) {
36         exp = 0;
37         while(n % primes[i] == 0) {
38             n /= primes[i];
39             exp++;
40         }
41         if(exp) {
42             aux = calcPeriod(primes[i], exp);
43             lcm = lcm * aux / gcd(lcm, aux);
44         }
45     }
46     if(n > 1) {
47         aux = calcPeriod(n, 1);
48         lcm = lcm * aux / gcd(lcm, aux);
49     }
50     return lcm;
51 }
52
53 void ident(ll a[2][2]) {
54     a[0][0] = a[1][1] = 1;
55     a[0][1] = a[1][0] = 0;
56 }
57
58 void multMat(ll a[2][2], ll b[2][2], ll c[2][2], ll mod) {
59     for(int i = 0; i < 2; i++) {
60         for(int j = 0; j < 2; j++) {
61             c[i][j] = (a[i][0] * b[0][j] + a[i][1] * b[1][j]) % mod;
62         }
63     }
64 }
65
66 void expbin(ll a[2][2], ll b[2][2], int n, int mod) {
67     ll c[2][2];
68     if(n < 0) return;
69     if(n == 0) ident(b);
70     else if(n & 1) {
71         expbin(a, c, n - 1, mod);
72         multMat(a, c, b, mod);
73     } else {
74         expbin(a, c, n >> 1, mod);
75         multMat(c, c, b, mod);
76     }
77 }
78
79 int main() {
80     ios::sync_with_stdio(0);
81     cin.tie(0);
82     int n, m, mod;
83     ll a[2][2], b[2][2];
84     a[0][0] = a[0][1] = a[1][0] = 1; a[1][1] = 0;
85     sieve();
86     while(cin >> n >> m) {
87         mod = findPeriod(m);

```



```

88     expbin(a, b, n % findPeriod(mod), mod);
89     expbin(a, b, b[0][1], m);
90     cout << b[0][1] << "\n";
91 }
92 return 0;
93 }

```

6.7 Fibonacci Factorization

```

1 // dada uma lista fib com uma quantidade "idx" de numeros de fibonacci, calcula
2 // uma possivel decomposiçao de um numero n em fatores de fibonacci (pode não
  existir tal fatoraço)
3
4 int flag = 0;
5 vector<ll> factors;
6
7 void factorize(ll n, int idx) {
8     if(flag) return;
9     if(n == 1) {
10         //fatores estarao em factors, processar a resposta aqui
11         flag = 1;
12         return;
13     }
14     for(int i = idx; i > 2; i--) {
15         if (n % fib[i] == 0) {
16             factors.push_back(i);
17             factorize(n / fib[i], i);
18             factors.pop_back();
19         }
20     }
21 }

```

6.8 Linear Sieve

```

1 int pi, lp[MAX], pr[MAX];
2
3 void linearSieve() {
4     pi = 0;
5     for(int i = 2; i < MAX; i++) {
6         if(lp[i] == 0) {
7             lp[i] = i;
8             pr[pi++] = i;
9         }
10        for(int j = 0; i * pr[j] < MAX; j++) {
11            lp[i * pr[j]] = pr[j];
12            if(pr[j] == lp[i]) break;
13        }
14    }
15 }
16
17 vector<int> factorize(int x) {
18     vector<int> factors;
19     while(x > 1) {
20         if(factors.empty() || factors.back() != lp[x]) {
21             factors.push_back(lp[x]);
22         }
23         x /= lp[x];
24     }

```

```

25     return factors;
26 }

```

6.9 Matrix Power

```

1 // Valores iniciais devem estar em m1
2 // O resultado final vai estar em m2
3 // Algoritmo não muito bom, ocupa 2 * log(k) * ord² memória
4
5 vector<vector<ll>> m1(MAXG, vector<ll> (MAXG)), m2(MAXG, vector<ll> (MAXG));
6
7 void multmat(vector<vector<ll>> &a, vector<vector<ll>> &b, vector<vector<ll>> &c
  , int ord) {
8     for(int i = 0; i < ord; i++) {
9         for(int j = 0; j < ord; j++) {
10             c[i][j] = 0;
11             for(int k = 0; k < ord; k++) {
12                 c[i][j] += (a[i][k] * b[k][j]) % MOD;
13                 c[i][j] %= MOD;
14             }
15         }
16     }
17 }
18
19 void ident(vector<vector<ll>> &a, int ord) {
20     for(int i = 0; i < ord; i++) {
21         for(int j = 0; j < ord; j++) {
22             a[i][j] = i == j;
23         }
24     }
25 }
26
27 void expbin(vector<vector<ll>> &a, vector<vector<ll>> &b, ll k, int ord) {
28     vector<vector<ll>> c(MAXG, vector<ll> (MAXG));
29     if(k < 0) return;
30     if(k == 0) ident(b, ord);
31     else if(k & 1) {
32         expbin(a, c, k - 1, ord);
33         multmat(a, c, b, ord);
34     } else {
35         expbin(a, c, k >> 1, ord);
36         multmat(c, c, b, ord);
37     }
38 }

```

6.10 Matrix Power Better

```

1 // Valores iniciais e resultado fica em m1
2 // Não ocupa muita memória extra
3
4 vector<vector<ll>> m1(MAXG, vector<ll> (MAXG)), m2(MAXG, vector<ll> (MAXG));
5
6 void multmat(vector<vector<ll>> &a, vector<vector<ll>> &b, int ord) {
7     vector<vector<ll>> c(MAXG, vector<ll> (MAXG));
8     for(int i = 0; i < ord; i++) {
9         for(int j = 0; j < ord; j++) {
10             for(int k = 0; k < ord; k++) {
11                 c[i][j] += (a[i][k] * b[k][j]) % MOD;

```

```

12         c[i][j] %= MOD;
13     }
14 }
15 }
16 a = c;
17 }
18
19 void ident(vector<vector<ll>> &a, int ord) {
20     for(int i = 0; i < ord; i++) {
21         for(int j = 0; j < ord; j++) {
22             a[i][j] = i == j;
23         }
24     }
25 }
26
27 void expbinmat(int k, int ord) {
28     ident(m2, ord);
29     while(k) {
30         if(k & 1) {
31             multmat(m2, m1, ord);
32         }
33         multmat(m1, m1, ord);
34         k >>= 1;
35     }
36     m1 = m2;
37 }

```

6.11 Mod Inverse

```

1 ll inv[MAX];
2
3 void modInverse() {
4     inv[0] = inv[1] = 1;
5     for(ll i = 2; i <= n; i++) {
6         inv[i] = ((MOD - MOD / i) * inv[MOD % i]) % MOD;
7     }
8 }

```

6.12 Phi

```

1 // função totiente de euler
2
3 int phi(int x) {
4     int exp, ans = 1;
5     for(int i = 0; i < pi; i++) {
6         if(primes[i] * primes[i] > x) break;
7         exp = 0;
8         while(x % primes[i] == 0) {
9             exp++;
10            x /= primes[i];
11        }
12        if(exp) {
13            ans *= (primes[i] - 1) * pow(primes[i], exp - 1);
14        }
15    }
16    if(x > 1) ans *= (x - 1);
17    return ans;
18 }

```

6.13 Pollards Rho

```
1 // a função fact retorna os fatores primos do numero, que podem ter repetições
2 // e não estar em ordem
3
4 ll mul(ll a, ll b, ll m) {
5     ll ret = a * b - ll((long double) 1 / m * a * b + 0.5) * m;
6     return ret < 0 ? ret + m : ret;
7 }
8
9 ll pow(ll x, ll y, ll m) {
10     if(!y) return 1;
11     ll ans = pow(mul(x, x, m), y / 2, m);
12     return y % 2 ? mul(x, ans, m) : ans;
13 }
14
15 bool prime(ll n) {
16     if(n < 2) return 0;
17     if(n <= 3) return 1;
18     if(n % 2 == 0) return 0;
19     ll r = __builtin_ctzll(n - 1), d = n >> r;
20     for(int a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) {
21         ll x = pow(a, d, n);
22         if(x == 1 || x == n - 1 || a % n == 0) continue;
23         for(int j = 0; j < r - 1; j++) {
24             x = mul(x, x, n);
25             if(x == n - 1) break;
26         }
27         if(x != n - 1) return 0;
28     }
29     return 1;
30 }
31
32 ll rho(ll n) {
33     if(n == 1 or prime(n)) return n;
34     auto f = [n](ll x) {return mul(x, x, n) + 1;};
35     ll x = 0, y = 0, t = 30, prd = 2, x0 = 1, q;
36     while(t % 40 != 0 or gcd(prd, n) == 1) {
37         if(x == y) x = ++x0, y = f(x);
38         q = mul(prd, abs(x - y), n);
39         if(q != 0) prd = q;
40         x = f(x), y = f(f(y)), t++;
41     }
42     return gcd(prd, n);
43 }
44
45 vector<ll> fact(ll n) {
46     if(n == 1) return {};
47     if(prime(n)) return {n};
48     ll d = rho(n);
49     vector<ll> l = fact(d), r = fact(n / d);
50     l.insert(l.end(), r.begin(), r.end());
51     return l;
52 }
```

6.14 Prime Test Sqrt

```

1 bool isPrime(int x) {
2     if(x <= 2) return false;
3     for(int i = 3; i * i <= x; i += 2) {
4         if(x % i == 0) return false;
5     }
6     return true;
7 }

```

6.15 Segmented Sieve

```

1 int pi;
2
3 vector<ll> sieve(int n) {
4     vector<ll> ans;
5     vector<bool> isPrime(n + 1, 1);
6     for(int i = 2; i * i <= n; i++) {
7         if(isPrime[i]) {
8             for(int j = i * i; j <= n; j += i) {
9                 isPrime[j] = 0;
10            }
11        }
12    }
13    for(int i = 2; i <= n; i++) {
14        if(isPrime[i]) ans.push_back(i);
15    }
16    return ans;
17 }
18
19 vector<ll> segmentedSieve(ll l, ll r) {
20     l = max(l, 2LL);
21     vector<bool> isPrime(r - l + 1, 1);
22     vector<ll> ans;
23     vector<ll> primes = sieve((int) sqrt(r) + 1);
24     for(ll p : primes) {
25         for(ll j = max(p * p, ((l + p - 1) / p) * p); j <= r; j += p) {
26             isPrime[j - l] = 0;
27         }
28     }
29     for(ll i = 0; i < r - l + 1; i++) {
30         if(isPrime[i]) {
31             ans.push_back(l + i);
32         }
33     }
34     pi = ans.size();
35     return ans;
36 }

```

6.16 Sieve

```

1 // cuidado com 0 e 1 no factorize, 0 vai rodar pra sempre, 1 vai ser confundido
  com primo
2
3 int pi;
4 ll primes[MAX];
5 bool isPrime[MAX];
6
7 void sieve() {
8     memset(isPrime, 1, sizeof(isPrime));

```

```

9      for(int i = 2; i * i < MAX; i++) {
10          if(isPrime[i]) {
11              for(int j = i * i; j < MAX; j += i) {
12                  isPrime[j] = 0;
13              }
14          }
15      }
16      pi = 0;
17      for(int i = 2; i < MAX; i++) {
18          if(isPrime[i]) primes[pi++] = i;
19      }
20 }
21
22 vector<pii> factorize(ll x) {
23     vector<pii> ans;
24     for(int i = 0; i < pi; i++) {
25         if(x % primes[i] == 0) {
26             int exp = 0;
27             while(x % primes[i] == 0) {
28                 exp++;
29                 x /= primes[i];
30             }
31             ans.emplace_back(primes[i], exp);
32         }
33         if(primes[i] * primes[i] > x) break;
34     }
35     if(x != 1) ans.emplace_back(x, 1);
36     return ans;
37 }
38
39 //Crivo número de divisores (n log n)
40 void numDivisors() {
41     for(int i = 1; i < MAX; i++) {
42         for(int j = i; j < MAX; j += i) {
43             arr[j]++;
44         }
45     }
46 }
47
48
49 //Crivo soma dos divisores (n log n)
50 //Cuidado com overflow
51 void sumDivisors() {
52     for(int i = 1; i < MAX; i++) {
53         for(int j = i; j < MAX; j += i) {
54             vet[j] += i;
55         }
56     }
57 }
58
59
60 //Crivo Totiente de Euler (n log log n)
61 void totient() {
62     for(int i = 1; i <= n; i++) arr[i] = i;
63     for(int i = 2; i <= n; i++) {
64         if(arr[i] == i) {
65             for(int j = i; j <= n; j += i) {

```

```

66         arr[j] -= arr[j] / i;
67     }
68 }
69 }
70 }
71
72
73 //Crivo maior divisor primo (n log log n)
74 int big[n + 1] = {1, 1};
75 for(int i = 1; i <= n; i++)
76     if(big[i] == 1)
77         for(int j = i; j <= n; j += i)
78             big[j] = i;

```

6.17 Stirling

```

1 //aproximação de stirling para a grandeza de n!
2
3 double stirling(int n){
4     return log(2. * PI * n) * 0.5 + n * (log(n) - 1.) ;
5 }
6
7 // Ou melhor:
8
9 ll stirling(ll n){
10     // expl(1.0L) = número de euler 2,7182...
11     if(n == 0) return 1;
12     long double pi = acosl(-1.L);
13     return (ll) (log10l(2.L * pi * n) * 0.5L + n * (log10l(n) - log10l(expl(1.0L
14         )))) + 1;

```

6.18 Subgrid Count

```

1 //Cuidado com possível overflow
2
3 //Conta quantos subgrids existem ((n + 1 escolhe 2) * (m + 1 escolhe 2))
4 ll totalSubgrids(ll n, ll m) {
5     return (n * (n + 1LL) / 2LL) * (m * (m + 1LL) / 2LL);
6 }
7
8 //Conta quantos subgrids de tamanho a x b existem (sem rotacionar);
9 //Somente quando a != b
10 ll fixedSubgrids(ll n, ll m, ll a, ll b) {
11     return (n - a + 1LL) * (m - b + 1LL);
12 }
13
14 //Conta quantos subgrids de tamanho a x b existem (com rotação);
15 //Somente quando a != b
16 ll rotFixedSubgrids(ll n, ll m, ll a, ll b) {
17     return (n - a + 1LL) * (m - b + 1LL) + (n - b + 1LL) * (m - a + 1LL);
18 }
19
20 //Conta quantos quadrados de lado k existem
21 ll totalSquares(ll m, ll n, ll k) {
22     ll s = min(n, m);
23     return s * (n + 1LL) * (m + 1LL) - ((n + m + 2LL) * s * (s + 1LL) / 2LL);
24 }

```

6.19 Sum Of Divisors

```
1 // retorna a soma de todos os divisores de x
2
3 ll sum(int x) {
4     ll ans = 1;
5     for(int i = 0; i < pi && primes[i] * primes[i] <= x; i++) {
6         if(x % primes[i] == 0) {
7             int exp = 0;
8             while(x % primes[i] == 0) {
9                 exp++;
10                x /= primes[i];
11            }
12            ans *= (pow(primes[i], exp + 1) - 1) / (primes[i] - 1);
13        }
14    }
15    if(x > 1) ans *= (pow(x, 2) - 1) / (x - 1);
16    return ans;
17 }
```

7 Strings

7.1 Aho Corasick

```
1 #define to_i(ch) ((ch) >= 'a' ? (ch) - 'a' + 26 : (ch) - 'A')
2
3 struct node {
4     int nxt[52], term = 0, p, pc, link = -1, go[52], ext = -1, occ = 0;
5     vector<int> idx;
6     node(int _p = 0, int _pc = 0) : p(_p), pc(_pc) {
7         memset(nxt, -1, sizeof(nxt));
8         memset(go, -1, sizeof(go));
9     }
10 };
11
12 vector<node> aca; //aho-corasick automaton
13 int occ[MAX]; //every word in the dict
14
15 void ins(const string &s, int idx) {
16     int u = 0;
17     for(int i = 0; i < s.size(); i++) {
18         int c = to_i(s[i]);
19         if(aca[u].nxt[c] == -1) {
20             aca[u].nxt[c] = aca.size();
21             aca.emplace_back(u, c);
22         }
23         u = aca[u].nxt[c];
24     }
25     aca[u].term = 1;
26     aca[u].idx.push_back(idx);
27 }
28
29 int go(int u, int c);
30
31 int link(int u) {
32     if(aca[u].link != -1) return aca[u].link;
33     return aca[u].link = (u == 0 || aca[u].p == 0) ? 0 :
34         go(link(aca[u].p), aca[u].pc);
35 }
```



```

35 }
36
37 int ext(int u) { //assumes u != 0
38     int v = link(u);
39     if(aca[u].ext != -1) return aca[u].ext;
40     return aca[u].ext = (v == 0 || aca[v].term) ? v : ext(v);
41 }
42
43 int go(int u, int c) {
44     if(aca[u].go[c] != -1) return aca[u].go[c];
45     if(aca[u].nxt[c] != -1) return aca[u].go[c] = aca[u].nxt[c];
46     return aca[u].go[c] = u == 0 ? 0 : go(link(u), c);
47 }
48
49 void process(const string &t) {
50     int u = 0;
51     for(int i = 0; i < t.size(); i++) {
52         int c = to_i(t[i]);
53         u = go(u, c);
54         for(int v = u; v; v = ext(v)) {
55             aca[v].occ++;
56         }
57     }
58     for(u = 0; u < (int) aca.size(); u++) {
59         for(auto &idx : aca[u].idx) {
60             occ[idx] += aca[u].occ;
61         }
62     }
63 }
64
65 void init() {
66     aca.clear();
67     aca.emplace_back();
68     fill(occ, occ + n, 0);
69     string t, s;
70     ins(s, i); // strings to be inserted(i = index 0 ~ queries-1);
71     process(t); // string to be searched upon
72     occ[i]; // occurrences of pattern s in text t
73 }

```

7.2 Kmp

```

1 string pat, tex;
2 int table[MAX]; // max size of string p;
3
4 void kmpPre() {
5     int i = 0, j = -1;
6     table[0] = -1;
7     while(i < m) {
8         while(j >= 0 && pat[i] != pat[j]) j = table[j];
9         table[++i] = ++j;
10    }
11 }
12
13 int kmp() {
14     int ans = 0, i = 0, j = 0;
15     kmpPre();
16     while(i < n) {

```

```

17     while(j >= 0 && tex[i] != pat[j]) j = table[j];
18     i++; j++;
19     if(j == m) ans++;
20 }
21 return ans;
22 }

```

7.3 Longest Palindromic Prefix

```

1 void lps(string str) {
2     string temp = str + '?';
3     reverse(str.begin(), str.end());
4     temp += str;
5     int n = temp.length();
6     vector<int> lp(n);
7     for(int i = 1; i < n; i++) {
8         int len = lp[i - 1];
9         while(len > 0 && temp[len] != temp[i]) {
10             len = lp[len - 1];
11         }
12         if(temp[i] == temp[len]) {
13             len++;
14         }
15         lp[i] = len;
16     }
17     cout << temp.substr(0, lp[n - 1]);
18 }

```

7.4 Manacher

```

1 void manacher(const string& s, vector<int>& d1, vector<int>& d2) {
2     int n = (int)s.size();
3     d1.assign(n, 0);
4     int l = 0, r = -1;
5     for(int i = 0; i < n; ++i) {
6         int k = 1;
7         if(i <= r) k = min(d1[l + r - i], r - i + 1);
8         while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
9         d1[i] = k;
10        if(i + k - 1 > r) { l = i - k + 1; r = i + k - 1; }
11    }
12    d2.assign(n, 0);
13    l = 0; r = -1;
14    for(int i = 0; i < n; ++i) {
15        int k = 0;
16        if(i <= r) k = min(d2[l + r - i + 1], r - i + 1);
17        while(i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) ++k;
18        d2[i] = k;
19        if(i + k - 1 > r) { l = i - k; r = i + k - 1; }
20    }
21 }
22
23 pair<int, pair<int,int>> longest_pal(const string& s) {
24     vector<int> d1, d2;
25     manacher(s, d1, d2);
26     int n = (int)s.size(), best = 0, L = 0, R = -1;
27     for(int i = 0; i < n; ++i) {
28         int len = 2 * d1[i] - 1;

```

```

29     if (len > best) { best = len; L = i - d1[i] + 1; R = i + d1[i] - 1; }
30 }
31 for(int i = 0; i < n; ++i) {
32     int len = 2 * d2[i];
33     if(len > best) { best = len; L = i - d2[i]; R = i + d2[i] - 1; }
34 }
35 return {best, {L, R}};
36 }
37
38 long long count_pal(const string& s) {
39     vector<int> d1, d2;
40     manacher(s, d1, d2);
41     long long ans = 0;
42     for(int x : d1) ans += x;
43     for(int x : d2) ans += x;
44     return ans;
45 }

```

7.5 Radix Trie

```

1 // Não sei se tá certo de verdade isso, principalmente a parte de contagem dos
  // prefixos (occ)
2 // temqueverissoae
3
4 #define to_i(ch) ((ch) - 'a')
5
6 struct node {
7     int nxt[26], occ = 0, term = 0;
8     string st;
9     node() {memset(nxt, -1, sizeof(nxt));}
10    node(string z) {memset(nxt, -1, sizeof(nxt)); st = z;}
11 };
12
13 vector<node> trie;
14
15 int match(string &s1, string &s2, int c) {
16     int ans = 0;
17     for(int i = 0; i < min(s2.size(), s1.size() - c); i++) {
18         if(s1[i + c] == s2[i]) ans++;
19         else break;
20     }
21     return ans;
22 }
23
24 void ins(string &s) {
25     int u = 0, i = 0, x, c, last;
26     trie[u].occ++;
27     while(i < s.size()) {
28         c = to_i(s[i]);
29         if(trie[u].nxt[c] == -1) {
30             trie[u].nxt[c] = trie.size();
31             trie.emplace_back(s.substr(i));
32             trie.back().term = 1;
33             trie.back().occ = 1;
34             return;
35         }
36         u = trie[u].nxt[c];
37         trie[u].occ++;

```

```

38     x = match(s, trie[u].st, i);
39     if(x < trie[u].st.size()) {
40         int old = trie[u].occ - 1;
41         last = trie.size();
42         trie.emplace_back(trie[u].st.substr(x));
43         trie[last].term = trie[u].term;
44         trie[last].occ = old;
45         memcpy(trie[last].nxt, trie[u].nxt, sizeof(trie[u].nxt));
46         memset(trie[u].nxt, -1, sizeof(trie[u].nxt));
47         trie[u].st = trie[u].st.substr(0, x);
48         trie[u].nxt[to_i(trie[last].st[0])] = last;
49         trie[u].term = i + x == s.size();
50     }
51     if(i + x == s.size()) {
52         trie[u].term = 1;
53         return;
54     }
55     i += x;
56 }
57 }
58
59 void search(string &s) {
60     int u = 0, i = 0, x;
61     while(i < s.size()) {
62         x = match(s, trie[u].st, i);
63         if(x == trie[u].st.size()) {
64             i += x;
65             if(trie[u].term == 1) {
66                 // Uma (ou mais) string com prefixo de tamanho i
67                 // em comum com s existe na trie
68             }
69             if(i < s.size() && trie[u].nxt[to_i(s[i])] != -1) {
70                 u = trie[u].nxt[to_i(s[i])];
71             } else return;
72         } else return;
73     }
74 }

```

7.6 Radix Trie Gpt

```

1 //Radix Trie do Gepeto, não testei isso
2
3 #define to_i(ch) ((ch)-'a')
4
5 struct node {
6     int nxt[26], occ, term;
7     string st;
8     node(): occ(0), term(0), st("") { memset(nxt, -1, sizeof(nxt)); }
9     node(string z): occ(0), term(0), st(z) { memset(nxt, -1, sizeof(nxt)); }
10 };
11
12 vector<node> trie;
13
14 void init_trie() {
15     trie.clear();
16     trie.emplace_back("");
17 }
18

```

```

19 int match(const string &s, const string &e, int c) {
20     int k = 0, n = (int)s.size(), m = (int)e.size();
21     while (k < m && c + k < n && s[c + k] == e[k]) ++k;
22     return k;
23 }
24
25 void ins(const string &s) {
26     int u = 0, i = 0;
27     trie[u].occ++;
28     while (i < (int)s.size()) {
29         int c = to_i(s[i]);
30         if (trie[u].nxt[c] == -1) {
31             trie[u].nxt[c] = (int)trie.size();
32             trie.emplace_back(s.substr(i));
33             trie.back().term = 1;
34             trie.back().occ = 1;
35             return;
36         }
37         u = trie[u].nxt[c];
38         trie[u].occ++;
39         int x = match(s, trie[u].st, i);
40         if (x < (int)trie[u].st.size()) {
41             int old = trie[u].occ - 1;
42             int last = (int)trie.size();
43             trie.emplace_back(trie[u].st.substr(x));
44             trie[last].term = trie[u].term;
45             trie[last].occ = old;
46             memcpy(trie[last].nxt, trie[u].nxt, sizeof(trie[u].nxt));
47             memset(trie[u].nxt, -1, sizeof(trie[u].nxt));
48             trie[u].st.resize(x);
49             trie[u].nxt[to_i(trie[last].st[0])] = last;
50             trie[u].term = (i + x == (int)s.size());
51             if (i + x == (int)s.size()) return;
52             i += x;
53             continue;
54         }
55         i += x;
56         if (i == (int)s.size()) { trie[u].term = 1; return; }
57     }
58 }
59
60 int count_prefix(const string &s) {
61     int u = 0, i = 0;
62     while (i < (int)s.size()) {
63         int x = match(s, trie[u].st, i);
64         if (i + x == (int)s.size()) return trie[u].occ; // prefixo termina
65         // aqui (mesmo no meio da aresta)
66         if (x < (int)trie[u].st.size()) return 0; // quebrou no meio
67         // sem terminar o prefixo
68         i += x;
69         int c = to_i(s[i]);
70         if (trie[u].nxt[c] == -1) return 0;
71         u = trie[u].nxt[c];
72     }
73     return trie[u].occ;
74 }

```

```

74 bool exists_word(const string &s) {
75     int u = 0, i = 0;
76     while (i < (int)s.size()) {
77         int x = match(s, trie[u].st, i);
78         if (x < (int)trie[u].st.size()) return false;
79         i += x;
80         if (i == (int)s.size()) return trie[u].term;
81         int c = to_i(s[i]);
82         if (trie[u].nxt[c] == -1) return false;
83         u = trie[u].nxt[c];
84     }
85     return false;
86 }

```

7.7 Trie

```

1 // MACRO PARA O ALFABETO TODO, a-z E A-Z
2 #define to_i(ch) ((ch) >= 'a' ? (ch) - 'a' + 26 : (ch) - 'A')
3
4 //MACRO PARA SOMENTE LETRAS MINUSCULAS (OU MAIUSCULAS, SÓ MUDAR O a PARA A)
5 #define to_i(ch) ((ch) - 'a')
6
7 struct node {
8     int nxt[52], term = 0, occ = 0, l = 0;
9     node() {memset(nxt, -1, sizeof(nxt));}
10 };
11
12 vector<node> trie;
13
14 void ins(string &s) {
15     int c1, u = 0;
16     for(auto c : s) {
17         c1 = to_i(c);
18         if(trie[u].nxt[c1] == -1) {
19             trie[u].l++;
20             trie[u].nxt[c1] = trie.size();
21             trie.emplace_back();
22         }
23         u = trie[u].nxt[c1];
24         trie[u].occ++;
25     }
26     trie[u].term = 1;
27 }
28
29 trie.clear();
30 trie.emplace_back();

```

8 Trees

8.1 All Dist Sum

```

1 // array ans contém para cada vértice, a soma de todas as distancias
2 // para os outros vértices
3
4 ll tsize[MAX], dist[MAX], prop[MAX], ans[MAX];
5
6 void dfs1(int u, int p) {
7     ll a = 0;
8     tsize[u] = 0;

```

```

9      for(int &v : graph[u]) {
10          if(v != p) {
11              dfs1(v, u);
12              a += prop[v];
13              tsize[u] += 1 + tsize[v];
14          }
15      }
16      dist[u] = a;
17      prop[u] = a + tsize[u] + 1;
18  }
19
20  void dfs2(int u, int p) {
21      if(u != 0) ans[u] = dist[u] + ans[p] - prop[u] + (n - tsize[u] - 1);
22      for(int &v : graph[u]) {
23          if(v != p) {
24              dfs2(v, u);
25          }
26      }
27  }
28
29  void compute() {
30      dfs1(0, 0);
31      ans[0] = dist[0];
32      dfs2(0, 0);
33  }

```

8.2 All Max Dist

```

1  // maxDist contém para cada vértice, a maior distancia para outro vértice na
   árvore
2
3  int mx, maxDist[MAX];
4
5  void dfs(int u, int p, int d) {
6      maxDist[u] = max(maxDist[u], d);
7      if(d > mx) mx = d, m = u;
8      for(int &v : graph[u]) {
9          if(v != p) dfs(v, u, d + 1);
10     }
11 }
12
13 void compute() {
14     dfs(0, -1, 0);
15     dfs(m, -1, 0);
16     dfs(m, -1, 0);
17 }

```

8.3 Binary Lifting

```

1  // dfs deve ser chamada com dfs(raiz, raiz, 0)
2
3  int depth[MAX], up[MAX][LOG];
4
5  void dfs(int u, int p, int d) {
6      up[u][0] = p;
7      depth[u] = d;
8      for(int i = 1; i < LOG; i++) {
9          up[u][i] = up[up[u][i - 1]][i - 1];

```

```

10     }
11     for(int &v : graph[u]) {
12         if(v != p) dfs(v, u, d + 1);
13     }
14 }
15
16 int findKthAncestor(int u, int k) {
17     if(k > depth[u]) return -1;
18     for(int i = 0; i < LOG; i++) {
19         if(k & (1 << i)) {
20             u = up[u][i];
21         }
22     }
23     return u;
24 }

```

8.4 Centroid

```

1 // Encontra o(s) centróide(s) da árvore (existem 1 ou 2 somente)
2 // Um centróide é um nodo que não possui nenhum filho com mais de floor(n / 2)
   filhos
3
4 int tsize[MAX];
5 vector<int> centroid;
6
7 void dfs(int u, int p) {
8     int flag = 0, s = 0;
9     for(int &v : graph[u]) {
10         if(v != p) {
11             dfs(v, u);
12             if(tsize[v] > n / 2) flag = 1;
13             s += tsize[v];
14         }
15     }
16     if(flag == 0 && n - s - 1 <= n / 2) {
17         centroid.push_back(u);
18     }
19     tsize[u] = s + 1;
20 }

```

8.5 Diameter

```

1 void dfs(int u, int p) {
2     for(int &v : graph[u]) {
3         if(v != p) {
4             arr[v] = arr[u] + 1;
5             dfs(v, u);
6         }
7     }
8 }
9
10 int diameter() {
11     int u = 0, v = 0;
12     arr[0] = 0;
13     dfs(0, 0);
14     for(int i = 0; i < n; i++) {
15         if(arr[i] > v) {
16             v = arr[i];

```



```

17         u = i;
18     }
19 }
20 arr[u] = 0;
21 dfs(u, u);
22 return *max_element(arr, arr + n);
23 }

```

8.6 Heavy Light Decomp

```

1 // Processa de forma eficiente operações dinâmicas e queries entre todos
2 // os vértices de um caminho entre quaisquer vértices u e v
3
4 #define left(u) (2 * (u))
5 #define right(u) (left(u) + 1)
6
7 int cur_pos, st[2 * MAX], parent[MAX], depth[MAX], heavy[MAX], head[MAX], pos[
8     MAX], aux[MAX];
9
10 int op(int a, int b) {
11     return max(a, b);
12 }
13
14 void build_st() {
15     for(int i = 0; i < n; i++) st[i + n] = aux[i];
16     for(int i = n - 1; i > 0; i--) {
17         st[i] = op(st[left(i)], st[right(i)]);
18     }
19 }
20
21 int query_st(int l, int r) {
22     int ans = 0;
23     for(l += n, r += n; l <= r; ++l /= 2, --r /= 2) {
24         if(l % 2 == 1) ans = op(ans, st[l]);
25         if(r % 2 == 0) ans = op(ans, st[r]);
26     }
27     return ans;
28 }
29
30 void update_st(int u, int x) {
31     u = pos[u];
32     st[u += n] = x;
33     while(u /= 2) st[u] = op(st[left(u)], st[right(u)]);
34 }
35
36 int dfs(int u) {
37     int sz = 1, v_sz, max_v_sz = 0;
38     for(int &v : graph[u]) {
39         if(v != parent[u]) {
40             parent[v] = u;
41             depth[v] = depth[u] + 1;
42             v_sz = dfs(v);
43             sz += v_sz;
44             if(v_sz > max_v_sz) {
45                 max_v_sz = v_sz;
46                 heavy[u] = v;
47             }
48         }
49     }
50 }

```

```

48     }
49     return sz;
50 }
51
52 void decompose(int u, int h) {
53     head[u] = h;
54     pos[u] = cur_pos++;
55     if(heavy[u] != -1) decompose(heavy[u], h);
56     for(int &v : graph[u]) {
57         if(v != parent[u] && v != heavy[u]) {
58             decompose(v, v);
59         }
60     }
61 }
62
63 void init() {
64     memset(heavy, -1, sizeof(heavy));
65     cur_pos = 0;
66     dfs(0);
67     decompose(0, 0);
68     for(int i = 0; i < n; i++) {
69         aux[pos[i]] = arr[i];
70     }
71     build_st();
72 }
73
74 int query(int u, int v) {
75     int ans = 0;
76     while(head[u] != head[v]) {
77         if(depth[head[u]] > depth[head[v]]) swap(u, v);
78         ans = op(ans, query_st(pos[head[v]], pos[v]));
79         v = parent[head[v]];
80     }
81     if(depth[u] > depth[v]) swap(u, v);
82     ans = op(ans, query_st(pos[u], pos[v]));
83     return ans;
84 }
85
86 // Corretude do algoritmo verificada com os problemas:
87 // https://cses.fi/problemset/task/2134/

```

8.7 Lca

```

1 // Calcula do lowest common ancestor de dois vértices com binary lifting
2 // dfs deve ser chamada com dfs(raiz, raiz, 0)
3
4 int depth[MAX], up[MAX][LOG];
5
6 void dfs(int u, int p, int d) {
7     up[u][0] = p;
8     depth[u] = d;
9     for(int i = 1; i < LOG; i++) {
10         up[u][i] = up[up[u][i - 1]][i - 1];
11     }
12     for(int &v : graph[u]) {
13         if(v != p) dfs(v, u, d + 1);
14     }
15 }

```

```

16
17 int findKthAncestor(int u, int k) {
18     if(k > depth[u]) return -1;
19     for(int i = 0; i < LOG; i++) {
20         if(k & (1 << i)) {
21             u = up[u][i];
22         }
23     }
24     return u;
25 }
26
27 int lca(int u, int v) {
28     if(depth[u] > depth[v]) swap(u, v);
29     v = findKthAncestor(v, depth[v] - depth[u]);
30     for(int i = LOG - 1; i >= 0; i--) {
31         if(up[u][i] != up[v][i]) {
32             u = up[u][i];
33             v = up[v][i];
34         }
35     }
36     return u != v ? up[u][0] : u;
37 }

```

8.8 Path Sum Update Static

```

1 // Faz update de soma em todos os vértices no caminho
2 // entre um par de vértices u e v qualquer
3 // Estático, todos os updates devem acontecer antes de qualquer consulta
4
5 // ATENÇÃO, A RAIZ DA ÁRVORE DEVE SER UM VÉRTICE 0 EXTRA, se a árvore já contém
6 // outra raiz 1, por exemplo, 1 deve ser pendurado em baixo de 0.
7 // isso significa que a indexação dos vértices deve ser 1-indexed
8
9 // LANÇAR dfs(0, 0) APÓS OS UPDATES, então sum[u] para qualquer u conterá sua
10 // soma final
11 int depth[MAX], up[MAX][LOG], diff[MAX], sum[MAX];
12
13 void dfs(int u, int p, int d) {
14     up[u][0] = p;
15     depth[u] = d;
16     for(int i = 1; i < LOG; i++) {
17         up[u][i] = up[up[u][i - 1]][i - 1];
18     }
19     for(int &v : graph[u]) {
20         if(v != p) dfs(v, u, d + 1);
21     }
22 }
23
24 int findKthAncestor(int u, int k) {
25     if(k > depth[u]) return -1;
26     for(int i = 0; i < LOG; i++) {
27         if(k & (1 << i)) {
28             u = up[u][i];
29         }
30     }
31     return u;
32 }

```

```

33
34 int lca(int u, int v) {
35     if(depth[u] > depth[v]) swap(u, v);
36     v = findKthAncestor(v, depth[v] - depth[u]);
37     for(int i = LOG - 1; i >= 0; i--) {
38         if(up[u][i] != up[v][i]) {
39             u = up[u][i];
40             v = up[v][i];
41         }
42     }
43     return u != v ? up[u][0] : u;
44 }
45
46 int dfs(int u, int p) {
47     sum[u] = diff[u];
48     for(int &v : graph[u]) {
49         if(v != p) {
50             sum[u] += dfs(v, u);
51         }
52     }
53     return sum[u];
54 }
55
56 void preprocess() {
57     // 1 ou qualquer outro vértice é a raiz original da árvore
58     graph[0].push_back(1);
59     graph[1].push_back(0);
60     dfs(0, 0, 0);
61 }
62
63 void update(int u, int v, int delta) {
64     int l = lca(u, v);
65     diff[u] += delta;
66     diff[v] += delta;
67     diff[l] -= delta;
68     diff[up[l][0]] -= delta;
69 }

```

8.9 Subtree Queries

```

1 // Constrói o euler tour da árvore
2 // A sub árvore enraizada em qualquer nodo u vai estar em um subarray
3 // delimitado por [tin[u], tout[u]]
4 // inicializar timer em 1 caso for utilizar bit para queries nos subarrays
5 // deixar 0 caso usar segtree
6
7 int timer, tin[MAX], tout[MAX];
8
9 void dfs(int u, int p) {
10     tin[u] = timer++;
11     for(int &v : graph[u]) {
12         if(v != p) dfs(v, u);
13     }
14     tout[u] = timer - 1;
15 }

```

8.10 Tarjan Offline Lca

```

1 int nodeRank[MAX], ancestor[MAX], vis[MAX], ans[MAX];

```

```

2  vector<vector<pii>> queries(MAX);
3
4  int findSet(int u) {
5      if(ancestor[u] == u) return u;
6      return ancestor[u] = findSet(ancestor[u]);
7  }
8
9  void unionSets(int u, int v) {
10     u = findSet(u);
11     v = findSet(v);
12     if(nodeRank[u] > nodeRank[v]) {
13         ancestor[v] = u;
14     } else {
15         ancestor[u] = v;
16         if(nodeRank[v] == nodeRank[u]) nodeRank[v]++;
17     }
18 }
19
20 void dfs(int u) {
21     vis[u] = 1;
22     ancestor[u] = u;
23     for(int v : graph[u]) {
24         if(vis[v] == 0) {
25             dfs(v);
26             unionSets(u, v);
27             ancestor[findSet(u)] = u;
28             ancestor[u] = u;
29         }
30     }
31     for(auto [v, i] : queries[u]) {
32         if(vis[v] == 1) {
33             ans[i] = ancestor[findSet(v)];
34         }
35     }
36 }
37
38 void processQueries(int q) {
39     for(int i = 0; i < q; i++) {
40         cin >> u >> v;
41         u--; v--;
42         queries[u].emplace_back(v, i);
43         queries[v].emplace_back(u, i);
44     }
45     dfs(0); //vertice raiz da arvore
46 }
47
48 void clean() {
49     fill(vis, vis + n, 0);
50     fill(nodeRank, nodeRank + n, 0);
51     for(int i = 0; i < n; i++) queries[i].clear();
52 }

```

8.11 Tree Isomorphism

```

1  // Retorna true se duas árvores são isomórficas
2
3  int c, degree[MAX], center[2];

```

```

4  vector<vector<int>> graph1(MAX);
5  vector<vector<int>> graph2(MAX);
6
7  void treeCenter(vector<vector<int>> &graph) {
8      queue<int> leaves;
9      int u, q2, q = 0, counter = 0;
10     c = 0;
11     for(u = 0; u < n; u++) {
12         degree[u] = graph[u].size();
13         if(degree[u] == 1) {
14             leaves.push(u);
15             counter++;
16             q++;
17         }
18     }
19     while(counter < n) {
20         q2 = 0;
21         for(int i = 0; i < q; i++) {
22             u = leaves.front();
23             leaves.pop();
24             for(int &v : graph[u]) {
25                 degree[v]--;
26                 if(degree[v] == 1) {
27                     leaves.push(v);
28                     counter++;
29                     q2++;
30                 }
31                 degree[u] = 0;
32             }
33         }
34         q = q2;
35     }
36     while(!leaves.empty()) { center[c++] = leaves.front(); leaves.pop(); }
37 }
38
39 string encode(int u, int p, vector<vector<int>> &graph) {
40     vector<string> labels;
41     for(int &v : graph[u]) {
42         if(v != p) labels.push_back(encode(v, u, graph));
43     }
44     sort(labels.begin(), labels.end());
45     string result = "";
46     for(auto &s : labels) {
47         result += s;
48     }
49     return "(" + result + ")";
50 }
51
52 bool isomorphic() {
53     treeCenter(graph1);
54     int t1 = c;
55     string tree1 = encode(center[0], -1, graph1);
56     treeCenter(graph2);
57     if(t1 != c) return false;
58     for(int i = 0; i < c; i++) {
59         if(tree1 == encode(center[i], -1, graph2)) {
60             return true;

```

```

61     }
62 }
63 return false;
64 }

```

9 Examples

9.1 Longest Paren Substr

```

1 // dada uma sting contento apenas caracteres '(' e ')', encontra
2 // qual o tamanho da maior substring de parenteses validos, e quantas strings
3 // desse tamanho (não necessariamente iguais) existem
4
5 #include <bits/stdc++.h>
6 using namespace std;
7
8 #define MAX 1123456
9 #define LOG 20
10 #define PI acos(-1.)
11 #define MAXG 1123
12 #define INF 1123456789
13 #define INFLL 112345678911234567
14 #define EPS 1e-9
15 #define MOD 1000000007
16 typedef long long ll;
17 typedef pair<int, int> pii;
18 typedef tuple<int, int, int> tiii;
19
20 int n, m, k, arr[MAX];
21 ll vet[MAX];
22 bitset<MAX> b1;
23 vector<vector<int>> graph(MAX);
24
25 #define left(u) ((u) + 1)
26 #define right(u, m, l) ((u) + 2 * ((m) - (l) + 1))
27
28 int st[2 * MAX];
29
30 int build_st(int u, int l, int r) {
31     if(l == r) return st[u] = arr[l];
32     int m = (l + r) / 2;
33     return st[u] = min(build_st(left(u), l, m), build_st(right(u, m, l), m + 1,
34         r));
35 }
36
37 int search_min(int u, int l, int r, int i, int j, int val) {
38     if(r < i || l > j || st[u] >= val) return n;
39     if(l == r) {
40         if(arr[l] < val) return l;
41     }
42     int m = (l + r) / 2;
43     int lft = search_min(left(u), l, m, i, j, val);
44     if(lft < n) return lft;
45     else return search_min(right(u, m, l), m + 1, r, i, j, val);
46 }
47
48 int search_zero(int u, int l, int r, int i, int j, int val) {
49     if(r < i || l > j || st[u] > val) return -1;

```

```

49     if(l == r) {
50         if(arr[l] == val) return l;
51     }
52     int m = (l + r) / 2;
53     int rgt = search_zero(right(u, m, l), m + 1, r, i, j, val);
54     if(rgt > -1) return rgt;
55     else return search_zero(left(u), l, m, i, j, val);
56 }
57
58 int main() {
59     ios::sync_with_stdio(0);
60     cin.tie(0);
61     int a, b, c, t, u, v, w, f, q, ans;
62     ll la, lb, lc, lans;
63     string s1, s2;
64     char ch1, ch2;
65     cin >> s1;
66     a = 0;
67     n = s1.size();
68     for(int i = 0; i < n; i++) {
69         if(s1[i] == '(') a++;
70         else a--;
71         arr[i] = a;
72     }
73     ans = 0, c = 1;
74     build_st(0, 0, n - 1);
75     for(int i = 0; i < n; i++) {
76         if(s1[i] == '(') {
77             int j = search_min(0, 0, n - 1, i + 1, n - 1, arr[i] - 1);
78             int k = search_zero(0, 0, n - 1, i + 1, j - 1, arr[i] - 1);
79             if(k > i) {
80                 if(k - i + 1 > ans) {
81                     ans = k - i + 1;
82                     c = 1;
83                 } else if(k - i + 1 == ans) c++;
84             }
85         }
86     }
87     cout << ans << ' ' << c << '\n';
88     return 0;
89 }

```

9.2 Mo

```

1 //Problema muito difícil:
2 // dados dois arrays x e y de tamanho n e dadas k queries do tipo i, j, processar
3 // somatorio(1 até i, somatorio(1 até j, abs(x[i] - y[j])))
4 // números indo de 1 até 10^8, utilizado compressão de índices para ajudar
5 // BSIZE ótimo aqui: n / sqrt(k)
6
7 #include <bits/stdc++.h>
8 using namespace std;
9
10 #define MAX 212345
11 #define LOG 20
12 #define PI acos(-1.)
13 #define MAXG 1123
14 #define INF 1123456789

```



```

15 #define INFLL 112345678911234567
16 #define EPS 1e-9
17 #define MOD 1000000007
18 typedef long long ll;
19 typedef pair<int, int> pii;
20 typedef tuple<int, int, int> tiii;
21
22 int n, m, k, arr[MAX];
23 ll current, vet[MAX], ans[MAX], x[MAX], y[MAX], mpx[MAX], mpy[MAX], unmap[MAX];
24 bitset<MAX> b1;
25 vector<vector<int>> graph(MAX);
26
27 void compress() {
28     map<int, int> remap;
29     int idx = 0;
30     for(int i = 0; i < 2 * n; i++) {
31         remap[arr[i]] = 0;
32     }
33     for(auto &[key, val] : remap) {
34         unmap[idx] = key;
35         val = idx++;
36     }
37     for(int i = 0; i < n; i++) {
38         mpx[i] = remap[x[i]];
39         mpy[i] = remap[y[i]];
40     }
41 }
42
43
44 #define left(u) (2 * (u))
45 #define right(u) (left(u) + 1)
46
47 ll st[2][2 * MAX][2];
48
49 ll query(int f, int l, int r, ll val) {
50     ll ans = 0, sz = 0;
51     for(l += 2 * n, r += 2 * n; l <= r; ++l /= 2, --r /= 2) {
52         if(l % 2 == 1) {
53             ans += st[f][l][0];
54             sz += st[f][l][1];
55         }
56         if(r % 2 == 0) {
57             ans += st[f][r][0];
58             sz += st[f][r][1];
59         }
60     }
61     return llabs(sz * val - ans);
62 }
63
64 void update(int f, int u, ll delta, ll val) {
65     st[f][u += 2 * n][1] += delta;
66     st[f][u][0] = st[f][u][1] * val;
67     while(u /= 2) {
68         st[f][u][0] = st[f][left(u)][0] + st[f][right(u)][0];
69         st[f][u][1] = st[f][left(u)][1] + st[f][right(u)][1];
70     }
71 }
72

```

```

73 const int BSIZE = 1000;
74 vector<tiii> queries(MAX);
75
76 bool comp(const tiii a, const tiii b) {
77     auto [al, ar, ai] = a;
78     auto [bl, br, bi] = b;
79     if(al / BSIZE != bl / BSIZE) return a < b;
80     return (al / BSIZE & 1) ? (ar < br) : (ar > br);
81 }
82
83 void removex(int i) {
84     int j = mpx[i];
85     current -= query(1, 0, j - 1, x[i]) + query(1, j + 1, 2 * n - 1, x[i]);
86     update(0, j, -1, x[i]);
87 }
88
89 void addx(int i) {
90     int j = mpx[i];
91     current += query(1, 0, j - 1, x[i]) + query(1, j + 1, 2 * n - 1, x[i]);
92     update(0, j, 1, x[i]);
93 }
94
95 void removey(int i) {
96     int j = mpy[i];
97     current -= query(0, 0, j - 1, y[i]) + query(0, j + 1, 2 * n - 1, y[i]);
98     update(1, j, -1, y[i]);
99 }
100
101 void addy(int i) {
102     int j = mpy[i];
103     current += query(0, 0, j - 1, y[i]) + query(0, j + 1, 2 * n - 1, y[i]);
104     update(1, j, 1, y[i]);
105 }
106
107 ll getAns() {
108     return current;
109 }
110
111 void mo() {
112     sort(queries.begin(), queries.begin() + k, comp);
113     int l = -1, r = -1;
114     for(int i = 0; i < k; i++) {
115         auto &[ql, qr, j] = queries[i];
116         while(l > ql) {
117             removex(l);
118             l--;
119         }
120         while(l < ql) {
121             l++;
122             addx(l);
123         }
124         while(r > qr) {
125             removey(r);
126             r--;
127         }
128         while(r < qr) {
129             r++;

```

```

130         addy(r);
131     }
132     ans[j] = getAns();
133 }
134 }
135
136 int main() {
137     ios::sync_with_stdio(0);
138     cin.tie(0);
139     int a, b, c, t, u, v, w, f, q;
140     ll la, lb, lc, lans;
141     string s1, s2;
142     char ch1, ch2;
143     cin >> n;
144     for(int i = 0; i < n; i++) {
145         cin >> x[i];
146         arr[i] = x[i];
147     }
148     for(int i = 0; i < n; i++) {
149         cin >> y[i];
150         arr[i + n] = y[i];
151     }
152     compress();
153     cin >> k;
154     for(int i = 0; i < k; i++) {
155         cin >> a >> b;
156         a--; b--;
157         queries[i] = {a, b, i};
158     }
159     mo();
160     for(int i = 0; i < k; i++) {
161         cout << ans[i] << '\n';
162     }
163     return 0;
164 }

```

10 Tools

10.1 Case Generator

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  #define MAX 212345
5  #define MAXG 1123
6  #define INF 1123456789
7  #define INFLL 112345678911234567
8  #define EPS 1e-9
9  typedef long long ll;
10 typedef pair<int, int> pii;
11 typedef tuple<int, int, int> tiii;
12
13 int n, m, arr[MAX], grid[MAXG][MAXG];
14 ll pref[MAX];
15 vector<set<int>> graph(MAX);
16 vector<pii> ans;
17
18 int main() {
19     ios::sync_with_stdio(0);

```

```

20     cin.tie(0);
21     freopen("input.txt", "w", stdout);
22     for(int i = 0; i < 1000; i++) {
23         cout << 5 << '\n';
24
25         std::random_device rd; // Seed source
26         std::mt19937 gen(rd()); // Mersenne Twister engine
27         // Define the distribution range [l, r]
28         std::uniform_int_distribution<> dist(1, 20);
29         for(int i = 0; i < 5; i++) {
30             for(int j = 0; j < 5; j++) {
31                 int random_number = dist(gen);
32                 std::cout << random_number << ' ';
33             }
34             cout << '\n';
35         }
36     }
37     return 0;
38 }

```

10.2 Tree Generator

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  #define MAX 112345
5  #define MAXS 1123
6  typedef long long ll;
7  typedef pair<int, int> pii;
8  typedef tuple<ll, ll, int> tlli;
9  const ll MOD = 1000000007;
10
11 int n, m, arr[MAX], parent[MAX], nodeRank[MAX], tsize[MAX], vis[MAX];
12 ll pref[MAX];
13 vector<vector<int>> graph(MAX);
14
15 int findSet(int u) {
16     if(parent[u] == u) return u;
17     return parent[u] = findSet(parent[u]);
18 }
19
20 void unionSets(int u, int v) {
21     u = findSet(u);
22     v = findSet(v);
23     if(nodeRank[v] < nodeRank[u]) {
24         parent[v] = u;
25     } else {
26         parent[u] = v;
27         if(nodeRank[u] == nodeRank[v]) nodeRank[v]++;
28     }
29 }
30
31 void dfs(int u, int p) {
32     vis[u] = 1;
33     m++;
34     for(int &v : graph[u]) {
35         if(vis[v] == 1 && v != p) {
36             cout << "Has cycles!\n";

```

```

37         exit(0);
38     }
39     if(v != p) {
40         dfs(v, u);
41         tsize[u] += tsize[v];
42     }
43 }
44 tsize[u]++;
45 }
46
47 int main() {
48     ios::sync_with_stdio(0);
49     cin.tie(0);
50     freopen("../Main/input.txt", "w", stdout);
51     int flag;
52     ll t, d;
53     int u, v, q;
54     std::random_device dev;
55     std::mt19937 rng(dev());
56     std::uniform_int_distribution<std::mt19937::result_type> num1(1,1000000000);
57     std::uniform_int_distribution<std::mt19937::result_type> vn1(1,100000);
58     cout << 4 << '\n';
59     for(int cases = 0; cases < 4; cases++) {
60         memset(nodeRank, 0, sizeof(nodeRank));
61         memset(vis, 0, sizeof(vis));
62         memset(tsize, 0, sizeof(tsize));
63         n = 100000;
64         q = 100000;
65         iota(parent, parent + n + 1, 0);
66         cout << n << ' ' << q << '\n';
67         for(int i = 0; i < n; i++) {
68             cout << num1(rng) << (i == n - 1 ? '\n' : ' ');
69         }
70         std::uniform_int_distribution<std::mt19937::result_type> vert1(1,n);
71         for(int i = 0; i < 100000000; i++) {
72             u = vert1(rng);
73             v = vert1(rng);
74             if(findSet(u) != findSet(v)) {
75                 unionSets(u, v);
76                 graph[u].push_back(v);
77                 graph[v].push_back(u);
78                 cout << u << ' ' << v << '\n';
79             }
80         }
81         for(int i = 2; i <= n; i++) {
82             if(findSet(1) != findSet(i)) {
83                 unionSets(1, i);
84                 cout << 1 << ' ' << i << '\n';
85             }
86         }
87         m = 0;
88         dfs(1, -1);
89         if(m != n) cout << "Not a tree!\n";
90         for(int i = 0; i < q; i++) {
91             int u = vert1(rng);
92             std::uniform_int_distribution<std::mt19937::result_type> dist4(1,

```

```
        tsize[u]);
93         cout << u << ' ' << dist4(rng) << '\n';
94     }
95     for(int i = 0; i < MAX; i++) graph[i].clear();
96 }
97 return 0;
98 }
```